LONDON METROPOLITAN UNIVERSITY

islington college
(इस्लिङ्टन कलेज)

CS4051NI/CC4059NI Fundamentals of Computing

70% Individual Coursework

Final Submission

2024/25 Spring

Student Name: Kritika Basel

London Met ID: 24046774

College ID: NP01CP4A240157

Assignment Due Date: Wednesday, May 14, 2025

Assignment Submission Date: Wednesday, May 14, 2025

Word Count: 7756

Project File Links:

Onedrive Drive Link:	**24046774_kritika_basel (1)**

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a marks of zero will be awarded.

24046774_Kritika_Basel 1.docx

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:95737856

Submission Date
May 14, 2025, 4:31 AM GMT+5:45

Download Date
May 14, 2025, 4:32 AM GMT+5:45

File Name
24046774_Kritika_Basel 1.docx

File Size
39.4 KB

54 Pages

5,260 Words

27,607 Characters

turnitin Page 1 of 60 - Cover Page Submission ID trn:oid:::3618:95737856

18% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **71** Not Cited or Quoted 16%
  Matches with neither in-text citation nor quotation marks
- **4** Missing Quotations 1%
  Matches that are still very similar to source material
- **5** Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- **1** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
0% Publications
18% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- 71 Not Cited or Quoted 16%
  Matches with neither in-text citation nor quotation marks
- 4 Missing Quotations 1%
  Matches that are still very similar to source material
- 5 Missing Citation 1%
  Matches that have quotation marks, but no in-text citation
- 1 Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

6% Internet sources
0% Publications
18% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Internet	
www.geeksforgeeks.org		2%
2	Submitted works	
The Robert Gordon University on 2025-04-20		2%
3	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2025-03-17		1%
4	Internet	
www.drawio.com		1%
5	Submitted works	
islingtoncollege on 2025-05-13		<1%
6	Submitted works	
Softwarica College Of IT & E-Commerce on 2020-09-04		<1%
7	Submitted works	
Asia Pacific University College of Technology and Innovation (UCTI) on 2022-11-21		<1%
8	Submitted works	
islingtoncollege on 2025-05-12		<1%
9	Internet	
mieczyslawzyner.pl		<1%
10	Submitted works	
CTI Education Group on 2017-02-15		<1%

11 Submitted works
Capella University on 2025-01-20 <1%

12 Submitted works
Middlesex University on 2008-05-12 <1%

13 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2024-01-09 <1%

14 Submitted works
The Robert Gordon University on 2025-04-20 <1%

15 Submitted works
Informatics Education Limited on 2012-07-11 <1%

16 Submitted works
Indian Institute of Technology Patna on 2024-01-01 <1%

17 Submitted works
INTI Universal Holdings SDM BHD on 2025-04-05 <1%

18 Internet
www.chegg.com <1%

19 Submitted works
Lingnan University on 2024-11-30 <1%

20 Submitted works
University of Greenwich on 2016-04-28 <1%

21 Submitted works
Swinburne University of Technology on 2024-10-28 <1%

22 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2021-06-26 <1%

23 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-10-30 <1%

24 Submitted works
University of Northampton on 2025-01-17 <1%

25 Submitted works
University of Greenwich on 2021-02-02 <1%

26 Submitted works
University of Northampton on 2025-01-17 <1%

27 Submitted works
American Intercontinental University Online on 2009-05-28 <1%

28 Submitted works
Kaplan International Colleges on 2025-02-17 <1%

29 Submitted works
(school name not available) on 2006-08-27 <1%

30 Submitted works
Polytechnic Institute Australia on 2023-09-24 <1%

31 Submitted works
Study Group Australia on 2014-05-26 <1%

32 Submitted works
University of South Australia on 2012-08-27 <1%

33 Submitted works
Caledonian College of Engineering on 2013-05-20 <1%

34 Submitted works
Udayana University on 2023-12-15 <1%

35 Internet
github.com <1%

36 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-10-18 <1%

37 Submitted works
Baze University on 2017-01-06 <1%

38 Submitted works
Info Myanmar College on 2024-12-13 <1%

39 Submitted works

Mid-Kent College on 2024-12-13 <1%

40 Submitted works

Udayana University on 2023-12-14 <1%

41 Submitted works

University of London External System on 2013-04-22 <1%

Table of Contents

1.Introduction 1
1.1Explanation about the project 1
1.2. Aims & Objectives 2
1.3. Tools used 3
1.3.1. Python IDLE 3
1.3.2. Word 3
1.3.3. Text Editor 4
1.3.4. Draw.io 4
2. Discussion and Analysis 5
2.1. Algorithm 5
2.2. Flow Chart 7
2.3. Pseudocode 10
2.3.1. Pseudocode for READ.py 10
2.3.2. Pseudocode for WRITE.py 11
2.3.3. Pseudocode for OPERATION.PY 14
2.3.4. Pseudocode for MAIN.py 21
2.4. Data Structure 24
2.4.1. Primitive Data Type 24
2.4.2. Collection Datatype 25
3. Program 28
3.1. Implementation of read.py 28
3.2. Implementation of WRITE.py 29
3.3. Implementation of operation.py 33
3.4. Implementation of MAIN.py 39
3.5. Output of selling to customer process 41
3.6. Output of restocking process 42
3.7. Creation of text file6 44
4. Testing 46
4.1. Test 1 : Showing the implementation of try, except 46
4.2. Test 2 : Selection purchase and sale of products 47
4.3. Test 3: File generation of purchase of product(s) (Purchasing multiple product(s)) 48

4.4. Test 4: File generation of sales process of product(s) (Selling multiple product(s)) 50
4.5. Test 5: Showing the update in the stock of product(s) 53
5. Conclusion 56
7. Appendix 59
7.1. Code of READ.py 59
7.2. Code of WRITE.py 61
7.3. Code of OPERATION.py 65
7.3. Code of MAIN.py 75

List of Tables

Table 1:TRY AND EXCEPT 46
Table 2:TESTING NEGATIVE AND NON EXISTED VALUE 47
Table 3:FILE GENERATION OF PURCHASE OF CUSTOMERS 48
Table 4:RESTOCK PRODUCTS 51
Table 5:UPDATE STOCK OF PRODUCTS 54

List of Figures

Figure 1: Python 3
Figure 2: MS Word 3
Figure 3:Main Flow Chart 7
Figure 4:FlowChart A 8
Figure 5:Flow Chart B 9
Figure 6: Int 24
Figure 7:String 24
Figure 8:list initialize 25
Figure 9:List Use case 26
Figure 10: Dictionaries 26
Figure 11:READ FILE (1) 28
Figure 12:READ FILE (2) 28
Figure 13: WRITE FILE (1) 29
Figure 14:WRITE FILE(2) 30
Figure 15:WRITE FILE (3) 31
Figure 16: WRITE FILE (4) 32
Figure 17: OPERATION FILE (1) 33
Figure 18:OPERATION FILE (2) 34
Figure 19:OPERATION FILE (3) 34
Figure 20: OPERATION FILE (4) 35
Figure 21: OPERATION FILE (5) 35
Figure 22: OPERATION FILE (6) 36
Figure 23: OPERATION FILE (7) 36
Figure 24:OPERATION FILE (8) 37
Figure 25:OPERATION FILE (9) 37
Figure 26:OPERATION FILE (10) 38
Figure 27:OPERATION FILE (11) 38
Figure 28:MAIN FILE (1) 39
Figure 29:MAIN FILE (2) 40
Figure 30:Selling to customers entry 41
Figure 31:Selling to customer Invoice 41

1.Introduction

1.1Explanation about the project

We care, is a skin care selling business where customers satisfaction is our top priority. Here we care about the customers and try to give our best to satisfy your needs and requirement. The program is developed by the business to help the customers work and daily life easier by giving them direct access to the store without being here physically. With the help of this developed program the product and be chosen with few clicks on the screen and will be delivered at your door with ease. Our website does not only save your time, but it also makes your shopping experience easy and smooth. With a range of skincare products to choose from, you can select and compare what suits you best at your convenience in the comfort of your home.

In this assigned individual coursework of “FUNDAMENTAL OD COMPUTING”, making a program to sell, buy goods using python program is the major objective. This program is assigned to be functional to make it user-friendly and easier for the customers to interact on. This program includes main, read, write and operation page which has different tasks of taking the user input what the customer wants to buy or sell and generate the invoice according to the purchase.it also includes a lot of logical thinking and challenges to develop an algorithm to make the program functional and user friendly. Additionally, the development cycle enhances critical thinking skills and reinforces the practical application of programming skills taught in class. This program comprises file handling, input/output operations, and logic-based flow, all of which are a close simulation of a transactional system attempting to deliver a seamless digital shopping experience to the user. Over the course and the lifetime of Application, we worked on the space of user interface design, structure of code and application performance. The aim was to have the software working but also one that was enjoyable and usable. The course work represented a good mix of technical skills and creativity that gave the students a solid experience in software development.

1.2. Aims & Objectives

Aims

The main aim of this coursework is development of functional program with the help of python programming, and to make the algorithm correct for the program to be user-friendly and usable, simple, clear, and helpful for users.

Objectives

This coursework helps us to learn more about the python program and enhance the skills and knowledge required. The major objective of this course work is:

- To create a simple program with Python that will help people buy and sell things.
- To learn how to take input from the users and display them what they want.
- To learn how to store, read, write with using txt files.
- To create a program that is user-friendly and is easy to use.
- To practice more of the basic skills of the Python program and coding better.
- To practice problem-solving and using deductive reasoning to make the program work correctly.
- To learn how to create invoices after purchasing for the customer.

1.3. Tools used

1.3.1. Python IDLE

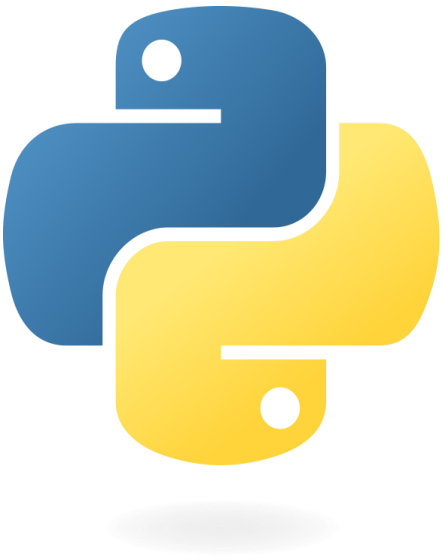


Figure 1: Python

Python is a programming language that is interpreted, Object-oriented, and considered to be high level. Python is one of the easiest yet most useful programming languages and is widely used in software industry. (GeeksforGeeks, 2023).

IDLE is used in this coursework to develop the actual program. With the help of python and IDLE the development of the program is being successful. The coding part of the coursework is done in the IDLE to develop a functional program. IDLE provides a simple and user-friendly environment that is especially helpful for writing, testing, and debugging Python code.

1.3.2. Word



Figure 2: MS Word

Microsoft Word is arguably the most widely used word processing program, famous for its flexibility and simplicity of use in document writing, editing, and formatting. Whether you're composing a simple letter, creating a resume, or producing a formal report MS-Word provides an enriched feature set to allow you to produce professional and well-organized content. (GeeksforGeeks,2021)

This software is used during the course to make the report part and to be able to explain the entire project in a structured manner.Word has helps to explain the coursework by breking down the complex part into smaller chunks and in well-mannered structure.

1.3.3. Text Editor



A basic Windows word processor is called Notepad. It allows computer users to compose documents. It is an easy text editing program on Windows. It has been included in all releases of Microsoft Windows since 1985 when Windows 1.0 was released. Typically, the resulting files are saved with the extension .txt. You owe only a responsibility to your operating system in terms of how you organize, copy, and store your notes with Notepad to maintain them (as text files) where you choose in the same location. By exporting your notes through any program for reading text files, you are at liberty to move them to another program.

1.3.4. Draw.io



Draw.io is a technology stack for building diagramming applications, and the world's most widely used browser-based end-user diagramming software.
draw.io is a registered trademark of JGraph Ltd and draw.io AG. JGraph Ltd is a company registered in England, draw.io AG is a company registered in Switzerland. Jointly, these companies develop and own the software, run the diagrams.net and draw.io sites and own the diagrams.net and draw.io brands. (draw.io, n.d.).

2. Discussion and Analysis

2.1. Algorithm

Step 1: Start

Step 2: Display the main menu

Step 3: Input number between 1-4

Step4: If number is 1 go to step 5

Step 5: Displays function of main menu go to step 2

Step 6: If number is 2 go to step 7.

Step 7: Input product id go to step 8

Step 8: If input is valid go to step 9 then go to step 11

Step9: Input quantity you want to buy Go to step 10

Step 10: If input quantity you want is valid go to step 12 then go to step 13

Step 11: Output enter a valid product id go to step 7

Step 12: Input if the user wants to buy more if yes go to step 7 then go to step 14

Step 13: Output invalid output or quantity out of stock go to step 9

Step 14: Input customer name go to step 15

Step 15: Input phone number go to step 16

Step 16: Display the invoice where bill_for_customer function is called and it writes all the purchase detail in a text file and displays in shell go to step 2

Step 17: If number is 3 go to step 18

Step 18: Input product id go to step 19

Step 19: If product id is valid go to step 20 then go to 21

Step 20: Input quantity go to 22

Step 21: Output Invalid id go to step 18

Step 22: If quantity is valid go to 23 then go to 24

Step 23: Input do you want to restock more if yes go to step 18 then go to step 25

Step 24: Output enter a valid input go to step20

Step 25: Input vendors name go to step 26

Step 26: Input phone number go to step 27

Step 27: Display the invoice where bill_for_vendors function is called and it writes all the restock detail in a text file and displays in shell goes to step 2

Step 28: if number is 4 go to step 29

Step 29: Input do you really want to exit if yes go to Step 30 then go to step 2

Step 30: Stop

2.2. Flow Chart

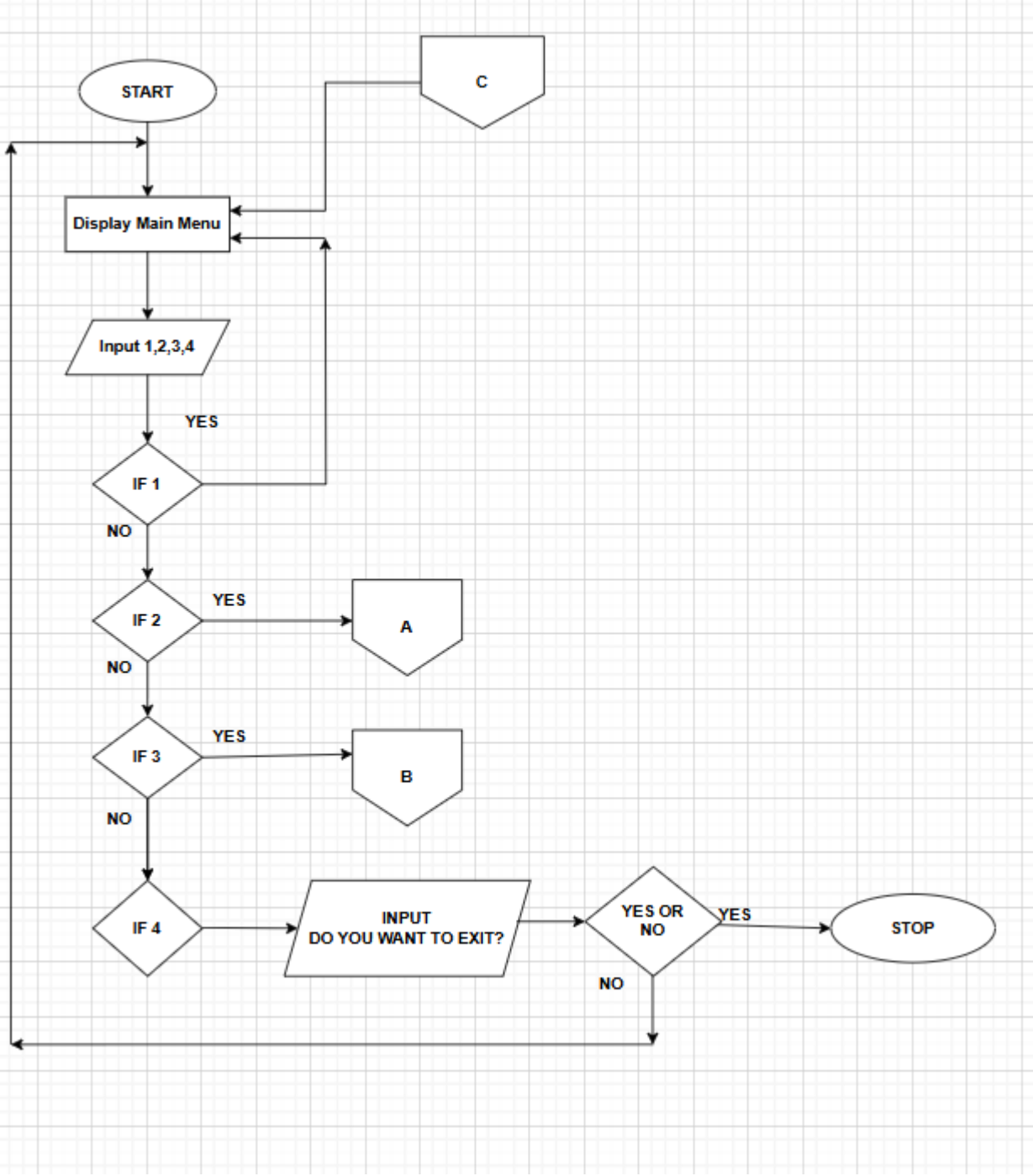


Figure 3:Main Flow Chart

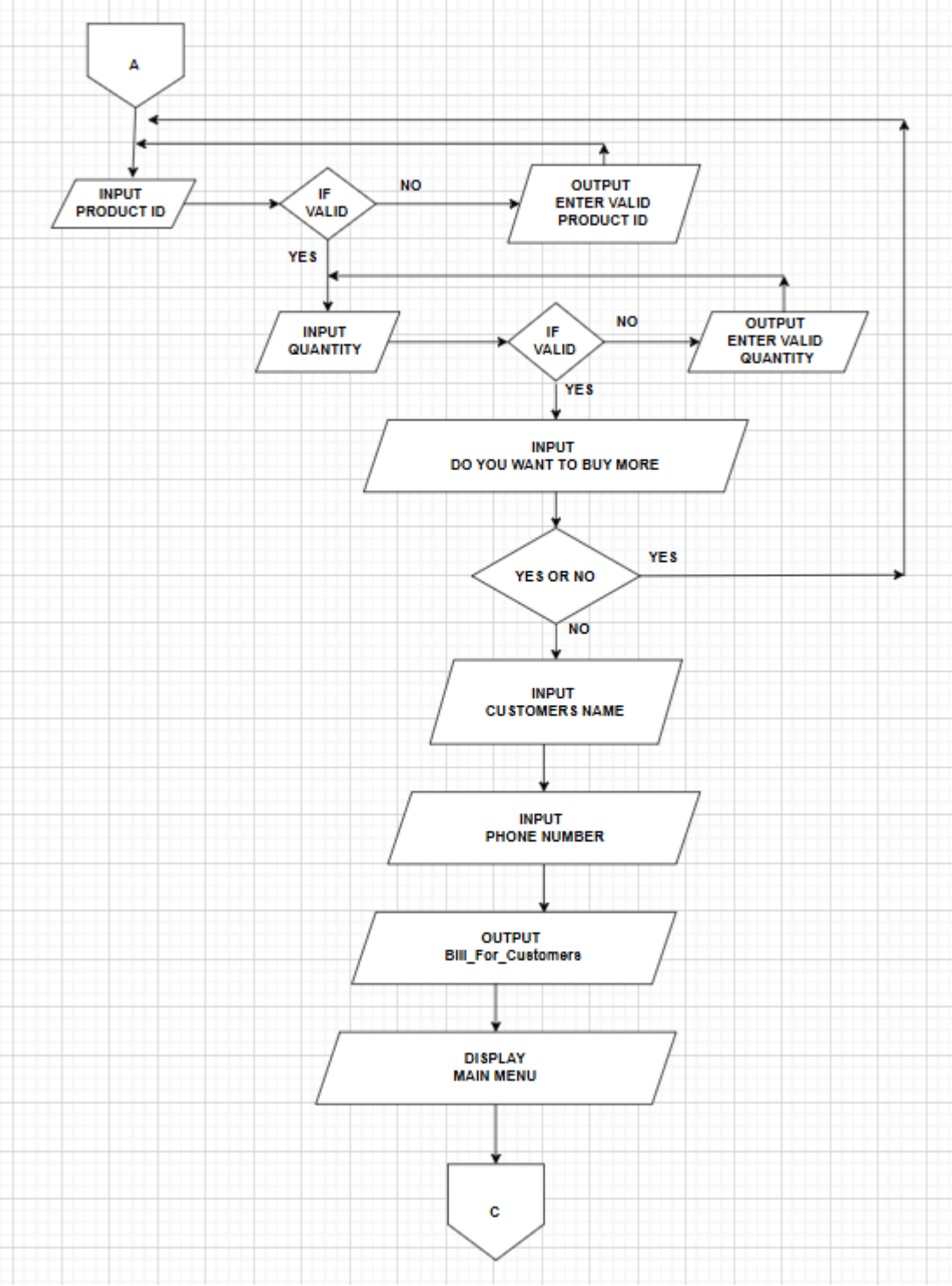


Figure 4:FlowChart A

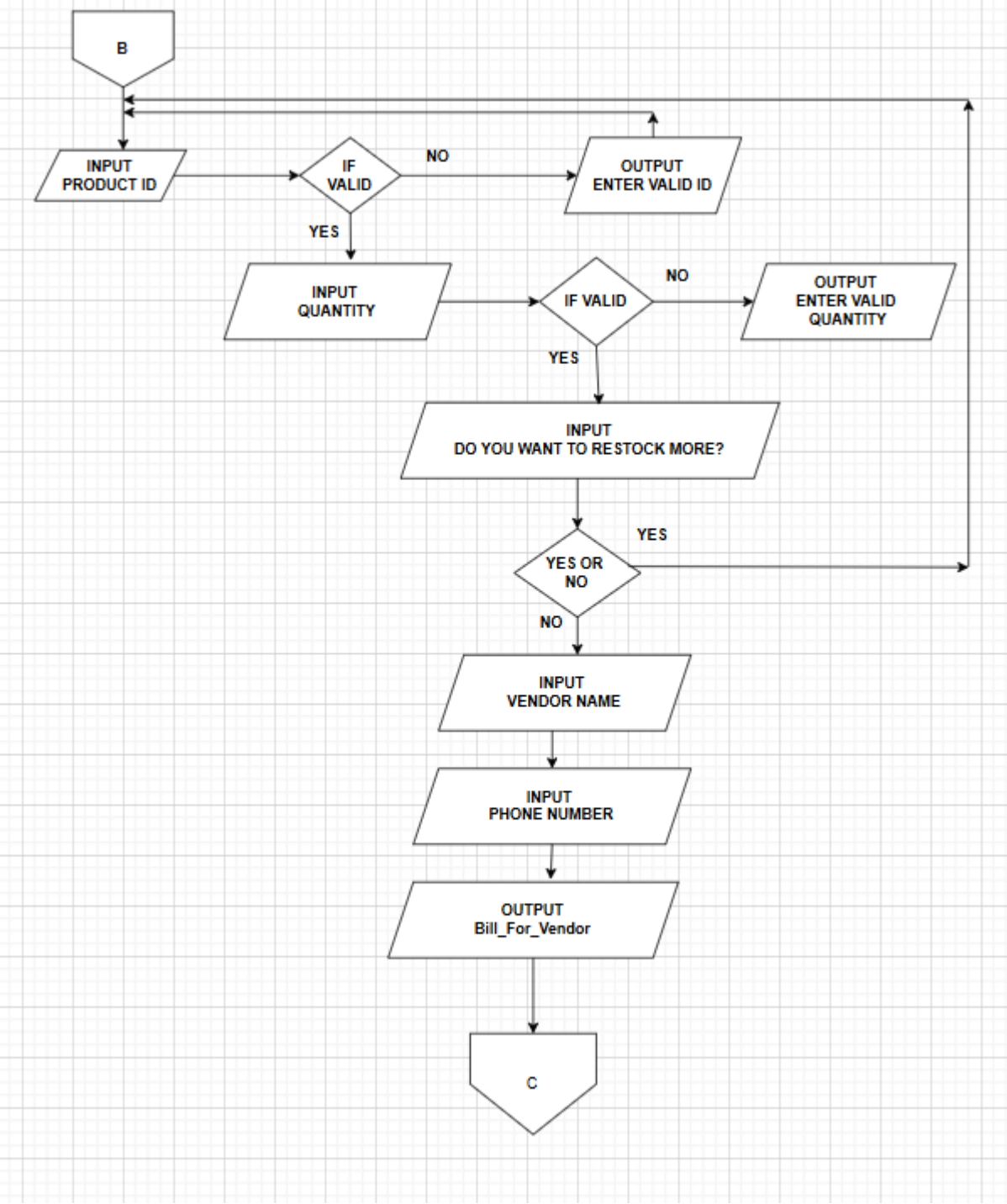


Figure 5:Flow Chart B

2.3. Pseudocode

2.3.1. Pseudocode for READ.py

```
ALGORITHM: READING

FUNCTION reading()

    '''Summary:
        this function reads the data from 'READ.txt', and displays
        it in a tabular format with calculated selling price
        and returns the data as a dictionary

    Parameters: None

    Returns:
        it returns disc_ where the key is product ID (int),
        and list containing:
        [Product Name(str), Brand(str), Available Quantity(str), Price(str), Country Of origin(str)].

    Raises:
        FileNotFoundError: if 'READ.txt' file does not exist.
    '''

    OUTPUT "-" * 110
    OUTPUT "ID", "\t", "Product Name", "\t\t", "Brand", "\t\t", "Available Quantity", "\t", "Price", "\t\t", "Country Of origin"
    OUTPUT "-" * 110

    WITH OPEN('READ.txt', 'r') AS file:
        SET store = file.read()
    END WITH

    SET listen = store.split('\n')
    file.close()
    SET disc_ = {}

    FOR i IN RANGE(LEN(listen)):
        SET disc_[i + 1] = listen[i].split(',')
    END FOR

    FOR key, value IN disc_.items():
        OUTPUT key, END='\t'

        FOR i IN RANGE(LEN(value)):
            IF i == 3:
```

```
            SET cost_price = int(value[i])
            SET selling_price = cost_price * 2
            OUTPUT selling_price, END='\t\t'
        END IF
        IF i == 0:
            OUTPUT value[i], END='\t\t'
        ELSE:
            OUTPUT value[i], END='\t\t'
        END IF
    END FOR

    OUTPUT NEW LINE
END FOR

OUTPUT '-' * 110

RETURN disc_

END FUNCTION

END ALGORITHM
```

2.3.2. Pseudocode for WRITE.py

```
ALGORITHM: bill_for_customers

FUNCTION bill_for_customers(items_bought, customer_name, phone_number,
amt_spent)

    '''Summary:
        This file generates and saves a text invoice of a customer
        if they purchase anything.
        The invoice is saved in txt file where customer details and all the
        necessary details are stored.

    Parameters:
        items_bought (list of dictionary): includes:
            'name'(str), 'quantity'(int), 'brand'(str), 'free'(int), 'price'(int), 'total'(int)
        customer_name (str): Customer's name
        phone_number (str): Customer's phone number
        amt_spent (int): Total amount spent

    Returns:
        str: filename of the saved invoice

    Raises:
```

```
        TypeError: if incorrect data types are passed
    '''

    SET now = datetime.datetime.now()
    SET date = str(now.year) + "-" + str(now.month) + "-" + str(now.day)
    SET filename = "Bill_for C " + customer_name + "_" + date + ".txt"

    WITH OPEN(filename, 'w') AS file:
        file.write("          === WE CARE BEAUTY STORE ===\n")
        file.write("\t   == INVOICE OF PURCHASE ==\n")
        file.write("\n")
        file.write("Customer: " + customer_name + "\t\t\tDate: " + date + "\n")
        file.write("Phone Number:" + phone_number + "\n")

        file.write("-" * 60)

        FOR item IN items_bought:
            file.write("\n")
            file.write("\tProduct" + "\t" * 4 + item['name'] + "\n")
            file.write("\tQuantity" + "\t" * 3 + str(item['quantity']) + "\n")
            file.write("\tBrand" + "\t" * 4 + item['brand'] + "\n")
            file.write("\tFree" + "\t" * 4 + str(item['free']) + "\n")
            file.write("\t Price" + "\t" * 4 + "Rs. " + str(item['price']))
            file.write("\n")
        END FOR

        file.write("-" * 60)
        file.write("\n")
        file.write("\tTotal" + "\t" * 5 + str(item['total']) + "\n")
        file.write("-" * 60)
        file.write("\n")
        file.write("\nInvoice saved as:" + filename)
        file.write("\n")
        file.write("Total Amount spent RS." + str(amt_spent) + "\n")
        file.write("ThankYou for shopping with us!\n")
    END WITH

    RETURN filename

END FUNCTION

END ALGORITHM
```

ALGORITHM: bill_for_vendors

```
FUNCTION bill_for_vendors(restocked_items, total_cost, supplier_name,
phone_number)

    '''Summary:
        This function generates and saves a text invoice
        to restock items in the store.
        This invoice includes supplier details, list of items restocked
        with price and quantity.

    Parameters:
        restocked_items (list of dict): includes:
            'name'(str), 'brand'(str), 'quantity'(str), 'cost_price'(int), 'total'(int)
        total_cost (int): total cost of all the items
        supplier_name (str): name of the supplier
        phone_number (str): phone number of the vendor

    Returns:
        str: filename of the invoice file

    Raises:
        TypeError: if parameters are passed with incorrect type
    '''

    SET now = datetime.datetime.now()
    SET date = str(now.year) + "-" + str(now.month) + "-" + str(now.day)
    SET filename = "Bill for S" + supplier_name + "_" + date + ".txt"

    WITH OPEN(filename, 'w') AS file:
        file.write("       === WE CARE BEAUTY STORE ===\n")
        file.write("\t    == INVOICE OF RESTOCK ==\n")
        file.write("Supplier Name : " + supplier_name + "\t\t\tDate: " + date + "\n")
        file.write("Phone: " + phone_number + "\n")
        file.write("-" * 75)

        FOR item IN restocked_items:
            file.write("\tProduct" + "\t" * 4 + item['name'] + "\n")
            file.write("\tBrand" + "\t" * 5 + item['brand'] + "\n")
            file.write("\tQty" + "\t" * 5 + str(item['quantity']) + "\n")
            file.write("\tUnit Price" + "\t" * 4 + "Rs. " + str(item['cost_price']))
            file.write("\n")
        END FOR

        file.write("\n")
        file.write("-" * 75)
```

```
        file.write("\tTotal" + "\t" * 4 + "Rs. " + str(item['total']) + "\n"))
        file.write("-" * 75)
        file.write("\n")
        file.write("Total Cost" + "Rs. " + str(item['total']))
        file.write("\nInvoice saved as:" + filename)
        file.write("Restocking Complete.\n")
    END WITH

    RETURN filename

END FUNCTION

END ALGORITHM
```

2.3.3. Pseudocode for OPERATION.PY

```
ALGORITHM: SELLING
IMPORT MODULE WRITE (bill_for_customers, bill_for_vendors)
IMPORT MODULE READ (reading)
IMPORT MODULE DATETIME

FUNCTION selling()
        '''
    Summary:
        This function handles the selling to the customers.It allows the user
        to purchase product and they can also get "buy 3 get 1 free"offer,
        updates the product,takes the necessary information fron the user and also
        displays the invoice using 'biil_for_customers' function.

    Parameters:
        None

    Returns:
        None

    Raises:
        ValueError:If non integer value is entered in product id or quantity.
        FileNotFoundError:If 'READ.txt' does not exist os is not accessible.
'''

SET all_the_products = reading()
SET items_bought = []
SET amt_spent = 0

WHILE TRUE
    TRY
```

```
    SET product_id = INT(INPUT("Enter the product ID that you want to buy (0 to exit):
"))
    IF product_id == 0
       BREAK
    END IF

    IF product_id NOT IN all_the_products
       OUTPUT "Enter a valid product id available!"
       CONTINUE
    END IF

    SET product = all_the_products[product_id]
    SET product_name = product[0]
    SET brand = product[1]
    SET available_qty = INT(product[2])
    SET price = INT(product[3]) * 2

    IF available_qty <= 0
       OUTPUT "This product is out of stock. Please choose another product."
       CONTINUE
    END IF

    WHILE TRUE
       TRY
          SET qty_to_buy = INT(INPUT("How much would you like to buy: "))
          IF qty_to_buy <= 0
             OUTPUT "Invalid quantity."
             CONTINUE
          END IF
          IF qty_to_buy > available_qty
             OUTPUT "The available quantity is:", available_qty
             CONTINUE
          END IF

          SET found = FALSE
          FOR item IN items_bought
             IF item['id'] == product_id
                SET item['quantity'] += qty_to_buy
                SET total_qty = item['quantity']
                SET item['free'] = total_qty // 3
                SET paid_items = total_qty + item['free']
                SET item['total'] = qty_to_buy * price
                SET found = TRUE
                BREAK
             END IF
          END FOR
```

```
            IF NOT found
                SET total_qty = qty_to_buy
                SET free_items = total_qty // 3
                SET paid_items = total_qty + free_items
                SET item_total = qty_to_buy * price

                CALL items_bought.APPEND({
                    'id': product_id,
                    'name': product_name,
                    'brand': brand,
                    'quantity': qty_to_buy,
                    'free': free_items,
                    'price': price,
                    'total': item_total
                })
            END IF

            SET new_qty = available_qty - paid_items
            IF new_qty < 0
                SET new_qty = 0
            END IF

            SET product[2] = STR(new_qty)
            SET amt_spent += item_total
            BREAK
        EXCEPT ValueError
            OUTPUT "Please enter a valid number!"
        END TRY
    END WHILE

    WITH OPEN('READ.txt', 'w') AS file
        FOR pid IN SORTED(all_the_products.keys())
            SET product = all_the_products[pid]
            CALL file.write(','.join(product) + '\n')
        END FOR
    END WITH

    WHILE TRUE
        SET again = INPUT("Do you want to buy more product? (yes/no): ").lower()
        IF again IN ('yes', 'no')
            BREAK
        END IF
        OUTPUT "Please enter 'yes' or 'no'"
    END WHILE
```

```
        IF again == 'no'
            BREAK
        END IF
    EXCEPT ValueError
        OUTPUT "Error: Please enter a valid number!"
        CONTINUE
    END TRY
END WHILE

IF NOT items_bought
    OUTPUT "\n No items were purchased."
    RETURN
END IF

OUTPUT "\n=== CUSTOMER DETAILS FOR INVOICE ==="
WHILE TRUE
    SET customer_name = INPUT("Enter customer name: ")
    IF customer_name.isalpha()
        BREAK
    ELSE
        OUTPUT "Name not valid."
    END IF
END WHILE

WHILE TRUE
    SET phone_number = INPUT("Enter phone number: ")
    IF phone_number.isdigit()
        BREAK
    ELSE
        OUTPUT "Enter a vaild phone number"
    END IF
END WHILE

SET filename = bill_for_customers(items_bought, customer_name, phone_number,
amt_spent)

OUTPUT "\n"
OUTPUT "\t\t  === Invoice of purchase ==="
OUTPUT "\n"
OUTPUT "Customer: " + customer_name + "\t\t\t", "Date: " +
STR(datetime.datetime.now())
OUTPUT "Phone: " + phone_number + "\n"
OUTPUT "-" * 65

FOR item IN items_bought
    OUTPUT "\tProduct", "\t"*4 + item['name']
```

```
    OUTPUT "\tQuantity", "\t"*4 + STR(item['quantity'])
    OUTPUT "\tBrand", "\t"*5 + item['brand']
    OUTPUT "\tFree", "\t"*5 + STR(item['free'])
    OUTPUT "\tUnit Price", "\t"*4 + "Rs. " + STR(item['price'])
END FOR

OUTPUT "-" * 65
OUTPUT "\tTotal", "\t"*5 + STR(item['total']) + "\n"
OUTPUT "-" * 65

OUTPUT "\nInvoice saved as:", filename
OUTPUT "Total amount: Rs.", amt_spent
OUTPUT "Thank you for your purchase!"
OUTPUT "\n"*5
END FUNCTION
END ALGORITHM



ALGORITHM: RESTOCKING
IMPORT MODULE WRITE(bill_for_customers, bill_for_vendors)
IMPORT MODULE READ (reading)
IMPORT MODULE DATETIME

FUNCTION restocking()

    '''
    Summary:
        This function handles the restocking preocess of the WE CARE.
        It allows the user to update quantity ,cost price and calculates vat
        and generates invoice using 'bill_for_vendors()' function.
    Parameters:
        None
    Returns:
        None
    Raises:
        ValueError:It is raised if non numeric input is provided in product id,quantity or
price.
        FileNotFound:If 'READ.txt' does not exist or is inaccessible.
'''

SET all_the_products = reading()
SET restocked_items = []
SET total_cost = 0

WHILE TRUE
```

```
    TRY
        SET product_id = INT(INPUT("Enter the product ID that you want to restock (0 to
exit): "))
        IF product_id == 0
            BREAK
        END IF

        IF product_id NOT IN all_the_products
            OUTPUT "Enter a valid product id available!"
            CONTINUE
        END IF

        SET product = all_the_products[product_id]
        SET product_name = product[0]
        SET brand = product[1]
        SET available_qty = INT(product[2])
        SET unit_cost = INT(product[3])

        WHILE TRUE
            TRY
                SET qty_to_add = INT(INPUT("How many units to add: "))
                IF qty_to_add <= 0
                    OUTPUT "Quantity should be positive!"
                    CONTINUE
                END IF

                SET new_qty = available_qty + qty_to_add
                SET product[2] = STR(new_qty)
                SET item_total = qty_to_add * unit_cost

                CALL restocked_items.APPEND({
                    'id': product_id,
                    'name': product_name,
                    'brand': brand,
                    'quantity': qty_to_add,
                    'unit_cost': unit_cost,
                    'total': total_cost
                })

                SET total_cost += item_total
                BREAK
            EXCEPT ValueError
                OUTPUT "Enter a valid number!"
            END TRY
        END WHILE
```

```
        WHILE TRUE
            SET again = INPUT("Do you want to restock more product? (yes/no): ").lower()
            IF again IN ('yes', 'no')
                BREAK
            END IF
            OUTPUT "Please enter 'yes' or 'no'"
        END WHILE

        IF again == 'no'
            BREAK
        END IF
    EXCEPT ValueError
        OUTPUT "Please enter a valid number!"
        CONTINUE
    END TRY
END WHILE

IF NOT restocked_items
    OUTPUT "\n No items were restocked."
    RETURN
END IF

WITH OPEN('READ.txt', 'w') AS file
    FOR pid IN SORTED(all_the_products.keys())
        SET product = all_the_products[pid]
        CALL file.write(','.join(product) + '\n')
    END FOR
END WITH

OUTPUT "\n=== VENDOR DETAILS FOR INVOICE ==="
WHILE TRUE
    SET vendor_name = INPUT("Enter vendor name: ")
    IF vendor_name.isalpha()
        BREAK
    ELSE
        OUTPUT "Name not valid."
    END IF
END WHILE

WHILE TRUE
    SET phone_number = INPUT("Enter vendor phone number: ")
    IF phone_number.isdigit()
        BREAK
    ELSE
        OUTPUT "Enter a valid phone number"
    END IF
```

```
END WHILE

SET filename = bill_for_vendors(restocked_items, vendor_name, phone_number,
total_cost)

OUTPUT "\n"
OUTPUT "\t\t  === Invoice of restock ==="
OUTPUT "\n"
OUTPUT "Vendor: " + vendor_name + "\t\t\t", "Date: " + STR(datetime.datetime.now())
OUTPUT "Phone: " + phone_number + "\n"
OUTPUT "-" * 65

FOR item IN restocked_items
   OUTPUT "\tProduct", "\t"*4 + item['name']
   OUTPUT "\tQuantity", "\t"*4 + STR(item['quantity'])
   OUTPUT "\tBrand", "\t"*5 + item['brand']
   OUTPUT "\tUnit Cost", "\t"*4 + "Rs. " + STR(item['unit_cost'])
END FOR

OUTPUT "-" * 65
OUTPUT "\tTotal", "\t"*5 + STR(item['total']) + "\n"
OUTPUT "-" * 65

OUTPUT "\nInvoice saved as:", filename
OUTPUT "Total cost: Rs.", total_cost
OUTPUT "Restocking completed successfully!"
OUTPUT "\n"*5
END FUNCTION
END ALGORITNM
```

2.3.4. Pseudocode for MAIN.py

```
ALGORITHM: Main Menu
IMPORT MODULE OPERATION (selling, restocking)
IMPORT MODULE READ (reading)

FUNCTION main_class()
   """
Summary:
   this function display the main menu of the WE CARE system and also
   handels user interaction.
   there are option to display,sell to customers,restock or exit the
   program.
```

```
Parameters:
   None

Returns:
   None

Raises:
   ValueError:If any non numeric values are given when number
   should be (1-4).
"""

WHILE True
   OUTPUT "*-*"*35
   OUTPUT "\n\t\t\t*=== ---------- WE CARE ---------- ===*"
   OUTPUT "*-*"*35
   OUTPUT "\n"
   OUTPUT "\t\t\t   WELCOME TO THE 'WE CARE' SKIN CARE COMPANY"
   OUTPUT "_"*105
   OUTPUT "\n"
   OUTPUT "\t\t  Insert one of the following command to connect with WE CARE"
   OUTPUT "\n"
   OUTPUT "1. Display the available products"
   OUTPUT "2. To Sell from WE CARE to customer"
   OUTPUT "3. To restock products from Supplier"
   OUTPUT "4. To exit from the program"

   SET user_choice = input("\nPlease choose an option (1-4): ")

   IF user_choice == '1'
      CALL reading()
   END IF

   IF user_choice == '2'
      CALL selling()
   END IF

   IF user_choice == '3'
      CALL restocking()
   END IF

   IF user_choice == '4'
      SET decision = input("Are you sure you want to exit? (yes/no): ").lower()
      IF decision == 'yes'
         OUTPUT "\n\n\nThank you for choosing WE CARE. Hope you had a great
experience here!!"
         BREAK
```

```
        END IF

        IF decision == 'no'
            CONTINUE
        END IF

        ELSE
            OUTPUT "Invalid input! Please enter 'yes' or 'no'"
        END IF
    END IF

    ELSE
        OUTPUT "\nInvalid choice! Please choose options 1-4"
    END IF

END WHILE

END FUNCTION

CALL main_class()

END ALGORITHM
```

2.4. Data Structure

2.4.1. Primitive Data Type

2.4.1.1 Int

Description

Int is the native Python type to represent values of integer (whole number). They are whole numbers with no decimal point and can be either positive, negative, or zero.

Purpose

Int data type is used here as the data stored to in the txt file is in string so the data can be accessed and be calculation can be done.

```
if i==3:
    num_=int((int(value[i])*200)/100)
```

Figure 6: Int

2.4.1.2. String

Description

String is a group of characters. Anything that is quoted in Python is a string. It contains numbers, letters, and symbols. Python do not have any data type for a character so one character is a string of length.

Purpose

In one of the case in this program, string is used while converting to write in text file.so in here too the integers are being converted into string.

```
file.write("\n")
file.write("\tProduct"+"\t"*4+item['name'] + "\n")
file.write( "\tQuantity"+"\t"*3+str(item['quantity']) + "\n" )
file.write( "\tBrand"+"\t"*4+item['brand']+ "\n")
file.write( "\tFree"+"\t"*4+ str(item['free']) + "\n" )
```

Figure 7:String

2.4.1.3. Float

Description

A float is a kind of data that is used to represent a number with a fraction part. It is used to store decimal numbers and math that needs to be precise.

2.4.1.4. Boolean

Description

Python Boolean type is among the data types that Python offers by default, which stores one of the two values i.e. True or False. It is typically used to store the truth values of the expressions.

2.4.2. Collection Datatype

2.4.2.1. List

Description

A list in Python is a dynamic sized array, native to Python. We can store all types of items in a list. List can contain mixed type of items this is because a list mainly stores reference at contiguous location and the actual items may be stored at other places.

Purpose

List is used here to store all the items bought do if there is same product being bought by one customer but separately then it helps to calculate and makes it easier.

```
all_the_products = reading()
items_bought = []  # This will store bought items
amt_spent = 0
```

Figure 8:list initialize

```
for item in items_bought:
    if item['id'] == product_id:
        item['quantity'] += qty_to_buy
        total_qty = item['quantity']
        item['free'] = total_qty // 3
        paid_items = total_qty + item['free']
        item['total'] = qty_to_buy * price
        found = True
        break
```

Figure 9:List Use case

2.4.2.2. Dictionaries

Description

Python dictionary is a data structure that stores the value in key: value pairs. The values in a dictionary can be of any data type and can be duplicated, whereas keys can't be repeated and should be immutable.

Purpose

The dictionary is used in here to store all the values in an id key of the dictionaries.

```
disc_={}

for i in range(len(listen)):
    disc_[i + 1]=listen[i].split(',')
```

Figure 10: Dictionaries

2.4.2.3. Tuple

Description

Python Tuple is a list of objects arranged in order, separated by commas. Python tuple is like a Python list in terms of indexing, nested objects, and repetition but the main difference between them is Python tuple is immutable, unlike the Python list which is mutable.

2.4.2.4. Set

Description

A Python set is an unordered collection of unique items. It is a data structure in Python to store greater than one item in a variable but cannot be a duplicate.Sets can be changed, meaning you can add or remove items after the set is created.

3. Program

3.1. Implementation of read.py

```
import datetime
'''this READ.py file allows all the file in this program to display the
available stock of the shop and displays the imformation to the user
for buying and restocking'''
def reading():
    '''Summary:
        this function reads the data from'READ.txt', and displays
        it in a tabular format with calculated seling price
        and returns the data as a dictionary

    Parameters:None

    Returns:
        it returns disc_ where the key is product ID(int),
        and list containing:
        [Product Name(str),Brand(str),Available Quantity(str),Price(str),Country Of origin(str)].
    Raises:
        FileNotFoundError:if 'READ.txt'file doesnot exist.

        '''
#read the existing document
    print("-" * 110)
    print("ID","\t","Product Name","\t\t","Brand","\t\t","Available Quantity","\t","Price","\t\t","Country Of origin")
    print("-" * 110)

#To read from the txt file
    with open('READ.txt', 'r') as file:
        store = file.read()

    listen = store.split('\n')#splits the column
    file.close()
```

Figure 11:READ FILE (1)

```
    file.close()
    disc_ = {}

    for i in range(len(listen)):
        disc_[i + 1] = listen[i].split(',')#splits according to ,.

    for key, value in disc_.items():
        print(key, end='\t')

        for i in range(len(value)):
            if i == 3:#the selling price calculation
                cost_price = int(value[i])
                selling_price = cost_price * 2
                print(selling_price, end='\t\t')
            elif i == 0:#to make the indentation correct
                print(value[i], end='\t\t')
            else:#to make the indentation correct
                print(value[i], end='\t\t')
        print()
    print('-' * 110)

    return disc_
```

Figure 12:READ FILE (2)

This is READ.py file, in this code the program is reading the data from a text file to display to the user in a cleaner and usable manner. In this code the function reading is

made which include all the code of the READ.py. Here, the data is read from the txt file then is being store in a dictionary and uses with the help of its keys. The selling price is also being calculated here while displaying the selling price should be there as it is for the customers to view.

3.2. Implementation of WRITE.py

```
import datetime
def bill_for_customers(items_bought,customer_name,phone_number,amt_spent):
    '''
    Summary:This file generate and saves a text invoice of a customer
    if they purchase anything.
    the invoice is saved in txt file where customer details and all the
    necessary details are stored.

    Parameters:
    items_bought(list of dictionary):In this list following information is included:
        'name'(Str):name of the product
        'quantity(int)':quantity purchased
        'brand'(str):brand of the product bought
        'free'(int):number of free item if got any
        'price'(int):price per item
        'total'(int):total price after free gifts
    customer_name(str):Customer's name
    phone_number(str):Cuctomer's phone number
    amt_apent(int):Total amount customer spent on total

    Returns:
    str:fileName (str) of the invoice which is stored on disk.

    Raises:
    TypeError:If incorrect data types are passes for any parameter.
'''
    now = datetime.datetime.now()
    date = str(now.year) + "-" + str(now.month) + "-" + str(now.day)
    fileName="Bill_for C "+customer_name+"_"+date+".txt"
```

Figure 13: WRITE FILE (1)

```
with open (fileName,'w')as file:
    #to print the invoice of customers
    file.write("           === WE CARE BEAUTY STORE ===\n")
    file.write("\t   == INVOICE OF PURCHASE ==\n")
    file.write("\n")
    file.write("Customer: " + customer_name + "\t\t\tDate: "+date+"\n" )
    file.write("Phone Number:"+phone_number+"\n")

    file.write("-"*60)
    #writing in the txt file
    for item in items_bought:
        #customers purchase
        file.write("\n")
        file.write("\tProduct"+"\t"*4+item['name'] + "\n")
        file.write( "\tQuantity"+"\t"*3+str(item['quantity']) + "\n" )
        file.write( "\tBrand"+"\t"*4+item['brand']+ "\n")
        file.write( "\tFree"+"\t"*4+ str(item['free']) + "\n" )
        file.write( "\t Price"+"\t"*4+"Rs. "+str(item['price']))
        file.write("\n")
    file.write("-"*60)
    file.write("\n")
    file.write("\tTotal"+"\t"*5 +str(item['total'])+"\n")
    file.write("-"*60)
    file.write("\n")
    #saving the invoice with the user's name and date
    file.write("\nInvoice saved as:"+fileName)
    file.write("\n")
    #displays how much amount is spent
    file.write("Total Amount spent RS."+ str(amt_spent) +"\n")
    file.write("ThankYou for shopping with us!\n")
```

Figure 14:WRITE FILE(2)

```
    return fileName

def bill_for_vendors(restocked_items,total_cost,supplier_name,phone_number):
    ''' Summary:
            This function generates and saves a text invoice
            to restock items in the store.
            this invoice includes supplier details,list of items restocked
            with its price and quantity.
        Parameters:
            restocked_items(lists of dict): it is list of dictionary
            which contains following information:
                'name'(str):name of the product
                'brand'(str):brand of the product
                'quantity'(str):quantity bought
                'cost_price'(int):original price
                'total'(int):total anount together
            total_cost(int):total cost of all the items
            supplier_name(str):name of the supplier]
            phone_number(str):phone number of vendor

        Returns:
            str:filename(str) of the invoice file is create and is saved in disks.
        Raises:
            TypeError:if any of the parameter is passes with incorrect type.

)

'''
    now = datetime.datetime.now()
```

Figure 15:WRITE FILE (3)

```
    filename="Bill for S"+ supplier_name+"_"+date+".txt"
#opens a new file in vendor's name and date
    with open(filename,'w')as file:
        #format for the display
        file.write("       === WE CARE BEAUTY STORE ===\n")
        file.write("\t   == INVOICE OF RESTOCK ==\n")
        file.write("Supplier Name : " + supplier_name + "\t\t\tDate: " +date+"\n")
        file.write("Phone: " + phone_number + "\n")
        file.write("-"*75)
        for item in restocked_items:
            #necessary information to display  restock
            file.write("\n")
            file.write("\tProduct"+"\t"*4+item['name'] + "\n")
            file.write("\tBrand"+"\t"*5  +item['brand'] + "\n")
            file.write("\tQty"+"\t"*5+ str(item['quantity']) + "\n" )
            file.write("\tUnit Price" +"\t"*4+"Rs. " + str(item['cost_price']))
            file.write("\n" )
        file.write("-"*75)
        file.write("\n")
        file.write("\tTotal"+"\t"*4+"Rs. " + str(item['total'])+"\n")
        file.write("-"*75)
        file.write("\n")
        #total cost spent
        file.write("Total Cost"+"Rs. " + str(item['total']))
        # writes saves the invoice as
        file.write("\nInvoice saved as:"+filename)
        file.write("\n")
        file.write("Restocking Complete.\n")

    return filename
```

Figure 16: WRITE FILE (4)

The WRITE.py is code in which the invoice is being created. The invoice for both customer and vendor are created separately. But in the write file there are two functionsbill_for_customers and bill_for_vendors. Each function helps to generate bills in a text file. The task of these functions is to get the data from the stored list through the

passed parameter and display it in a new txt file created under the name of the customer or vendor.

3.3. Implementation of operation.py

```
from READ import reading
import datetime
from WRITE import bill_for_customers, bill_for_vendors

def selling():
    '''
    Summary:
        This function handles the selling to the customers.It allows the user
        to purchase product and they can also get "buy 3 get 1 free"offer,
        updates the product,takes the necessary information fron the user and also
        displays the invoice using 'biil_for_customers' function.

    Parameters:
        None

    Returns:
        None

    Raises:
        ValueError:If non integer value is entered in product id or quantity.
        FileNotFoundError:If 'READ.txt' does not exist os is not accessible.
'''
    all_the_products = reading()
    items_bought = []  # This will store bought items
    amt_spent = 0

    while True:
        try:
            #take input of the product id
            product_id = int(input("Enter the product ID that you want to buy (0 to exit): "))
```

Figure 17: OPERATION FILE (1)

```
try:
    #take input of the product id
    product_id = int(input("Enter the product ID that you want to buy (0 to exit): "))

    if product_id == 0:
        break

    if product_id not in all_the_products:
        print("Enter a valid product id available!")
        continue

    product = all_the_products[product_id]
    product_name = product[0]
    brand = product[1]
    available_qty = int(product[2])
    price = int(product[3]) * 2  # Adjusted for price logic

    # Check if product is out of stock
    if available_qty <= 0:
        print("This product is out of stock. Please choose another product.")
        continue

    while True:
        try:
            qty_to_buy = int(input("How much would you like to buy: "))
            #checks the quantity is vaild or available or not.
            if qty_to_buy <= 0:
                print("Invalid quantity.")
                continue
            if qty_to_buy > available_qty:
                print("The available quantity is:", available_qty)
```

Figure 18:OPERATION FILE (2)

```
                print("The available quantity is:", available_qty)
                continue
            #for the item alredy inserted
            found = False
            for item in items_bought:
                if item['id'] == product_id:
                    item['quantity'] += qty_to_buy
                    total_qty = item['quantity']
                    item['free'] = total_qty // 3
                    paid_items = total_qty + item['free']
                    item['total'] = qty_to_buy * price
                    found = True
                    break
            #if it is not found update here
            if not found:
                total_qty = qty_to_buy
                free_items = total_qty // 3
                paid_items = total_qty + free_items
                item_total = qty_to_buy * price

                # Update the product info in the list
                items_bought.append({
                    'id': product_id,
                    'name': product_name,
                    'brand': brand,
                    'quantity': qty_to_buy,
                    'free': free_items,
                    'price': price,
                    'total': item_total
                })
```

Figure 19:OPERATION FILE (3)

```
                # Updating the quantity in the inventory
                new_qty = available_qty - paid_items
                if new_qty < 0:
                    new_qty = 0
                product[2] = str(new_qty)

                amt_spent += item_total
                break

            except ValueError:
                print("Please enter a valid number!")

        with open('READ.txt', 'w') as file:
            for pid in sorted(all_the_products.keys()):
                product = all_the_products[pid]
                # Write back to file in the original format
                file.write(','.join(product) + '\n')

        while True:
            #if the user want to continue buying
            again = input("Do you want to buy more product? (yes/no): ").lower()
            if again in ('yes', 'no'):
                break
            print("Please enter 'yes' or 'no'")

        if again == 'no':
            break

    except ValueError:
        print("Error: Please enter a valid number!")
```

Figure 20: OPERATION FILE (4)

```
            except ValueError:
                print("Error: Please enter a valid number!")
                continue

        if not items_bought:
            print("\n No items were purchased.")
            return

    # Get customer details for invoice
        print("\n=== CUSTOMER DETAILS FOR INVOICE ===")
        while True:
            customer_name = input("Enter customer name: ")
            if(customer_name.isalpha()):
                break
            else:
                print("Name not valid.")
        while True:
            phone_number = input("Enter phone number: ")
            if(phone_number.isdigit()):
                break
            else:
                print("Enter a vaild phone number")

        filename=bill_for_customers(items_bought,customer_name,phone_number,amt_spent)
        print("\n")
        print("\t\t  === Invoice of purchase ===")
        print("\n")
        print("Customer: " + customer_name + "\t\t\t","Date: " + str(datetime.datetime.now()) )

        print("Phone: " + phone_number + "\n")
        print("-" * 65)
```

Figure 21: OPERATION FILE (5)

```
    print("-" * 65)
    for item in items_bought:
        print("\tProduct","\t"*4+item['name'] + "\n" + "\tQuantity","\t"*4+str(item['quantity']) + "\n" + "\tBrand","\
              "\tUnit Price","\t"*4+"Rs. "+str(item['price']))
    print("-" * 65)
    print("\tTotal","\t"*5 +str(item['total'])+"\n")
    print("-" * 65)

    print("\nInvoice saved as:", filename)
    print("Total amount: Rs.", amt_spent)
    print("Thank you for your purchase!")
    print("\n"*5)

def restocking():
    '''
    Summary:
        This function handles the restocking preocess of the WE CARE.
        It allows the user to update quantity ,cost price and calculates vat
        and generates invoice using 'bill_for_vendors()' function.
    Parameters:
        None
    Returns:
        None
    Raises:
        ValueError:It is raised if non numeric input is provided in product id,quantity or price.
        FileNotFound:If 'READ.txt' does not exist or is inaccessible.
    '''
    products = reading()
    restocked_items = []
    total_cost = 0
```

Figure 22: OPERATION FILE (6)

```
    '''
    products = reading()
    restocked_items = []
    total_cost = 0

    while True:
        try:
            product_id = int(input("\nEnter product ID to restock (0 to finish): "))

            if product_id == 0:
                break
            if product_id not in products:
                print("Invalid product ID! Please enter a valid ID from the list.")
                continue

            product = products[product_id]
            current_qty = int(product[2])
            cost_price = int(product[3])   # Gets original cost price

            print("\nProduct: " + product[0])
            print("Brand: " + product[1])
            print("Current stock: " + str(current_qty))
            print("Cost price: Rs. " + str(cost_price))

            try:
                #Validation of the quantity is checked
                restock_qty = int(input("Enter quantity to add: "))
                if restock_qty==0:
                    break
                if restock_qty <= 0:
                    print("Quantity must be positive!")
```

Figure 23: OPERATION FILE (7)

```
            print("Quantity must be positive!")
            continue
        #to calculate new price if the price has changed
        new_cost_price = input("Enter new cost price (current: Rs. " + str(cost_price) + "), or press Enter to

        if new_cost_price:
            try:
                new_cost_price = int(new_cost_price)
                if new_cost_price <= 0:
                    print("Price must be positive! Keeping original price.")
                else:
                    cost_price = new_cost_price
                    product[3] = str(cost_price)
            except ValueError:
                print("Invalid price! Keeping original price.")

        # Calculate item total
        item_total = restock_qty * cost_price
        total_cost += item_total
        amt_after_vat=(total_cost*0.13)+total_cost

        # Add to restocked items list
        restocked_items.append({
            'name': product[0],
            'brand': product[1],
            'quantity': restock_qty,
            'cost_price': cost_price,
            'total': total_cost
        })

        # Update quantity
```

Figure 24:OPERATION FILE (8)

```
                'total': item_total
            })

            # Update quantity
            product[2] = str(current_qty + restock_qty)

            # Write changes back to file
            with open('READ.txt', 'w') as file:
                for pid in sorted(products.keys()):
                    p = products[pid]
                    file.write(','.join(p) + '\n')

            print("\nSuccessfully added " + str(restock_qty) + " units of " + product[0])
            print("New stock level: " + product[2])

            while True:
                #if they want to restock more
                again = input("Do you want to RESTOCK more product? (yes/no): ").lower()
                if again in ('yes', 'no'):
                    break
                print("Please enter 'yes' or 'no'")

            if again == 'no':
                break
        except ValueError:
            print("Invalid quantity! Please enter a number.")

    except ValueError:
        print("Invalid input! Please enter a number.")
if not restocked_items:
    print("\n No items were RESTOCKED.")
```

Figure 25:OPERATION FILE (9)

```
if not restocked_items:
    print("\n No items were RESTOCKED.")
    return
#display format for invoice
print("\n=== SUPLIER DETAILS FOR INVOICE ===")
supplier_name = input("Enter Vendors name: ")
while True:
    phone_number = input("Enter phone number: ")
    if(phone_number.isdigit()):
        break
    else:
        print("Enter a valid number.")
#prints the invoice for supplier
if restocked_items:

    filename = bill_for_vendors(restocked_items,total_cost,supplier_name,phone_number)
    print("\n")
    print("\t\t\t=== SUPPLIERS DETAILS ===")
    print("\n")
    print("Supplier Name : " + supplier_name + "\t\t\t","Date: " + str(datetime.datetime.now()) )
    print("Phone: " + phone_number + "\n")

    print("-" * 75)
    for item in restocked_items:
        print("\tProduct","\t"*4+item['name'] + "\n" +"\tBrand","\t"*5  +item['brand'] + "\n" +"\tQty","\t"*5+ str
         "\tUnit Price" ,"\t"*4+"Rs. " + str(item['cost_price']))
    print("-" * 75)
    print("\tTotal","\t"*4+"Rs. " + str(item['total']))
    print("-" * 75)
```

Figure 26:OPERATION FILE (10)

```
filename = bill_for_vendors(restocked_items,total_cost,supplier_name,phone_number)
print("\n")
print("\t\t\t=== SUPPLIERS DETAILS ===")
print("\n")
print("Supplier Name : " + supplier_name + "\t\t\t","Date: " + str(datetime.datetime.now()) )
print("Phone: " + phone_number + "\n")

print("-" * 75)
for item in restocked_items:
    print("\tProduct","\t"*4+item['name'] + "\n" +"\tBrand","\t"*5  +item['brand'] + "\n" +"\tQty","\t"*5+ str
     "\tUnit Price" ,"\t"*4+"Rs. " + str(item['cost_price']))
print("-" * 75)
print("\tTotal","\t"*4+"Rs. " + str(item['total']))
print("-" * 75)
print("\n")
print("TOTAL COST: Rs.", total_cost)
#vat calculation
print("Total Cost After VAT",(str(amt_after_vat)))
print("\nInvoice saved as:", filename)
print("\nRestocking complete.")
print("\n" *15)
```

Figure 27:OPERATION FILE (11)

In the operation.py there are two functions. Selling is one of the functions which helps the customers to buy good from the store easier. The system asks for the valid product id, quantity from the customer and if the customer buys 3 product, then they get one more product for free. The selling function has parameters like id, name of the product, brand, quantity, price, etc. The selling function implements the logic for the customer to purchase from WE CARE.

Another function of the operation.py is restocking. This function helps the suppliers to restock in the shop as it also calculates vat and the bill is generated after both expression is done in this the txt file is also updated after every purchase and restock it increase or decrease the quantity available in stock. The restocking function has parameter like id , quantity , price, etc. Operation modules handle some exception if the product id, quantity is wrong.

3.4. Implementation of MAIN.py

```
from OPERATION import selling,restocking
from READ import reading

def main_class():
    """
    Summary:
        this function display the main menu of the WE CARE system and also
        handels user interaction.
        there are option to display,sell to customers,restock or exit the
        program.

    Parameters:
        None

    Returns:
        None

    Raises:
        ValueError:If any non numeric values are given when number
        should be (1-4).
"""
    while True:
        print("*-*"*35)
        print("\n\t\t\t*=== -------------- WE CARE ------------------ ===*")
        print("*-*"*35)
        print("\n")
        print("\t\t\t   WELCOME TO THE 'WE CARE' SKIN CARE COMPANY")
        print("_"*105)
        print("\n")
        print("\t\t  Insert one of the following command to connect with WE CARE")
        print("\n")
```

Figure 28:MAIN FILE (1)

```
        print("\t\t  Insert one of the following command to connect with WE CARE")
        print("\n")
        print("1. Display the available products")
        print("2. To Sell from WE CARE to customer")
        print("3. To restock products from Supplier")
        print("4. To exit from the program")

        user_choice = input("\nPlease choose an option (1-4): ")

        if user_choice == '1':#to display the file
            reading()
        elif user_choice == '2':#selling to the customers
            selling()
        elif user_choice == '3':#restoking from the vendor
            restocking()
        elif user_choice == '4':
            #user input to make sure
            decision = input("Are you sure you want to exit? (yes/no): ").lower()
            if decision == 'yes':
                print("\n\n\nThank you for choosing WE CARE. Hope you had a great experience here!!")
                break
            elif decision == 'no':
                continue
            else:
                print("Invalid input! Please enter 'yes' or 'no'")
        else:
            print("\nInvalid choice! Please choose options 1-4")

main_class()
```

Figure 29:MAIN FILE (2)

The Main.py is the file that need to be run, and it is the actual page interacting with the user. In the main file the reading, selling and restocking functions are imported as in the main function you must perform specific task according to the user input so everything should be connected to the main function. While displaying there are 4 options given to the user to select from1-4.if 1 is pressed then all the items available in store will be displayed, if 2 is pressed then the user can input the product id, quantity they want and they can purchase from the store, if 3 is pressed then the store can restock items they want and for the both selling and restocking invoices are generated. And lastly, if 4 is pressed you can finally exit from the store system.

3.5. Output of selling to customer process

```
                    WELCOME TO THE 'WE CARE' SKIN CARE COMPANY
--------------------------------------------------------------------------------

                Insert one of the following command to connect with WE CARE

1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 2
-------------------------------------------------------------------------------------------------------------
ID      Product Name            Brand           Available Quantity      Price           Country Of origin
-------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum         Garnier                 12              800             France
2       Skin Cleanser           Cetaphil                50              1000            Switzerland
3       Sunscreen               Aqualogica              259             400             India
4
5
-------------------------------------------------------------------------------------------------------------
Enter the product ID that you want to buy (0 to exit): 1
How much would you like to buy: 2
Do you want to buy more product? (yes/no): no

=== CUSTOMER DETAILS FOR INVOICE ===
Enter customer name: kritika
Enter phone number: 1234567890
```

Figure 30:Selling to customers entry

```
                    === Invoice of purchase ===


Customer: kritika                           Date: 2025-05-14 00:11:25.993717
Phone: 1234567890

-----------------------------------------------------------------
        Product                                 Vitamin C Serum
        Quantity                                2
        Brand                                    Garnier
        Free                                    0
        Unit Price                              Rs. 800
-----------------------------------------------------------------
        Total                                   1600

-----------------------------------------------------------------

Invoice saved as: Bill_for C kritika_2025-5-14.txt
Total amount: Rs. 1600
Thank you for your purchase!
```

Figure 31:Selling to customer Invoice

This is the selling process to the customer. At first it will display all the product available with its price ,quantity available so when the user chosses a valid product id then it asks for a valid quantity and if the user wants to buy mre then the process repeats and if the

user doesnot want to buy the system asks for customers name and phone number by which it generates invoice for the purchase.

3.6. Output of restocking process

```
                Insert one of the following command to connect with WE CARE

1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 3
-------------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand           Available Quantity      Price           Country Of origin
-------------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum          Garnier                 10              800             France
2       Skin Cleanser            Cetaphil                50              1000            Switzerland
3       Sunscreen                Aqualogica              259             400             India
4
-------------------------------------------------------------------------------------------------------------------

Enter product ID to restock (0 to finish): 1

Product: Vitamin C Serum
Brand:  Garnier
Current stock: 10
Cost price: Rs. 400
Enter quantity to add: 20
Enter new cost price (current: Rs. 400), or press Enter to keep same:

Successfully added 20 units of Vitamin C Serum
```

Figure 32:OUTPUT OF RESTOCK

```
New stock level: 30
Do you want to RESTOCK more product? (yes/no): no

=== SUPLIER DETAILS FOR INVOICE ===
Enter Vendors name: rdfgth
Enter phone number: 1234567890

                        === SUPPLIERS DETAILS ===


Supplier Name : rdfgth                  Date: 2025-05-14 00:20:25.428915
Phone: 1234567890

--------------------------------------------------------------------------------
        Product                                 Vitamin C Serum
        Brand                                    Garnier
        Qty                                     20
        Unit Price                              Rs. 400
--------------------------------------------------------------------------------
        Total                           Rs. 8000
--------------------------------------------------------------------------------


TOTAL COST: Rs. 8000
Total Cost After VAT 9040.0

Invoice saved as: Bill for Srdfgth_2025-5-14.txt

Restocking complete.
```

Figure 33:SUPPLIER INVOICE

In the restocking process, at first the system asks for a valid product id then quantity and if both are correct it lets you buy the product from the store and then it asks for vendors name and phone number from where the products are being restocked. Then with the help of that it generates an invoice in the shells and runs the program as usual.

3.7. Creation of text file

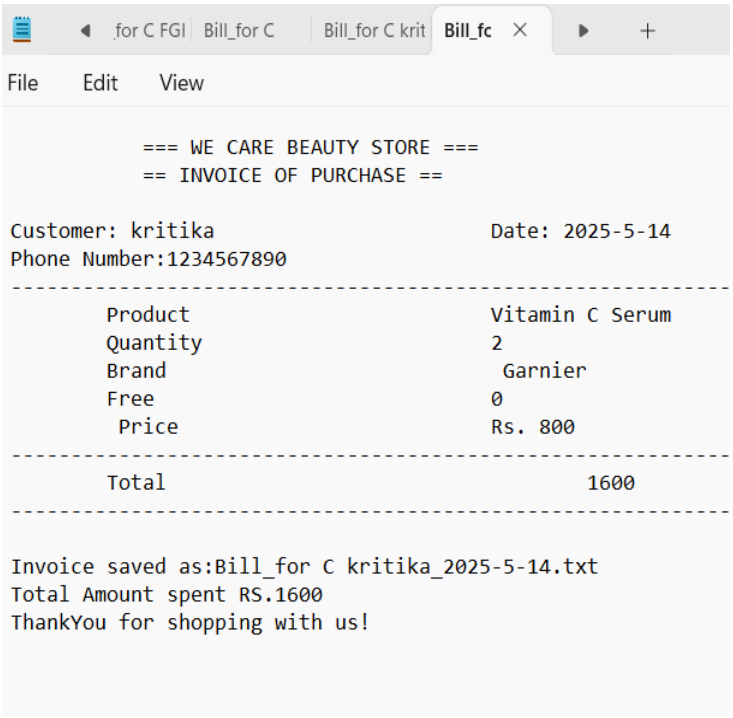


Figure 34:INVOICE IN OF PURCHASE IN .TXT

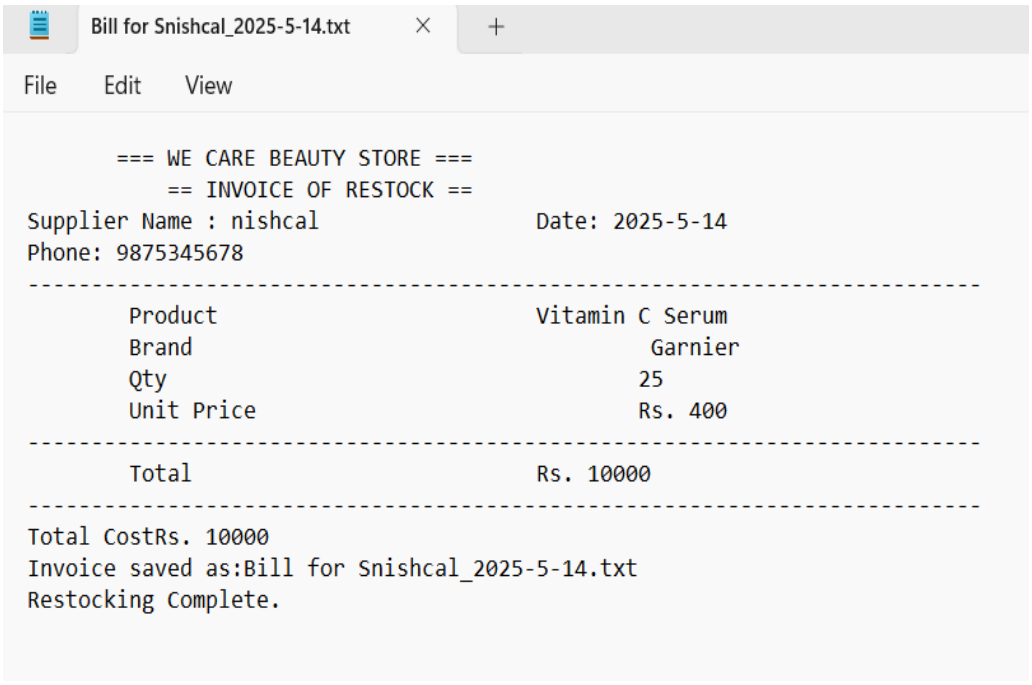


Figure 35:INVOICE OF PURCHSE IN .TXT

To generate an invoice, you must either sell something to a customer or just restock the items of the store. This invoice is generated when I purchased from the store and when all the input I gave was valid then the items were purchased and then the invoice is generated in both txt and shell. And the same thing will happen if you want to restock the stock in the system.

4. Testing

4.1. Test 1 : Showing the implementation of try, except

Table 1:TRY AND EXCEPT

Objective	To test the implementation of try and except
Action	• Run the program • Enter 2 to sell product to customers • Enter an alphabetical product id.
Expected Result	Error message invalid choice would be shown.
Actual Result	Error message invalid choice showed.
Conclusion	The implementation of try and except test was successful.

```
                    WELCOME TO THE 'WE CARE' SKIN CARE COMPANY
_____________________________________________________________________________________________________


                Insert one of the following command to connect with WE CARE


1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 2
-------------------------------------------------------------------------------------------------------------
ID       Product Name           Brand          Available Quantity      Price          Country Of origin
-------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum         Garnier               30              800            France
2       Skin Cleanser           Cetaphil              50              1000           Switzerland
3       Sunscreen               Aqualogica            259             400            India
4
5
-------------------------------------------------------------------------------------------------------------
Enter the product ID that you want to buy (0 to exit): a
Error: Please enter a valid number!!!
Enter the product ID that you want to buy (0 to exit):
```

Figure 36:NON NUMERIC VALUE

4.2. Test 2 : Selection purchase and sale of products

Table 2:TESTING NEGATIVE AND NON EXISTED VALUE

Objective	To test the implementation on selling and restoking by providing negative and non-existed value.
Action	For purchase • Run the program select 2 at first • Enter -1 in the product id • Enter 1 in the product id • Enter 30 in the quantity For sale • Enter 3 to show the options • Enter -1 in product id • Enter 1 in the product id • Enter -2 in the quantity
Expected Result	Error message should be shown.
Actual Result	Error message showed.
Conclusion	The test was successful.

```
                Insert one of the following command to connect with WE CARE

1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 2
-------------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand           Available Quantity      Price           Country Of origin
-------------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum          Garnier                 29              800             France
2       Skin Cleanser            Cetaphil                50              1000            Switzerland
3       Sunscreen                Aqualogica              259             400             India
4
-------------------------------------------------------------------------------------------------------------------
Enter the product ID that you want to buy (0 to exit): -1
Enter a valid product id available!
Enter the product ID that you want to buy (0 to exit): 1
How much would you like to buy: 30
The available quantity is: 29
How much would you like to buy:
```

Figure 37:NEGATIVE VALUE WHILE SELLING

```
1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 3
-------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand          Available Quantity      Price          Country Of origin
-------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum          Garnier               28             800            France
2       Skin Cleanser            Cetaphil              50             1000           Switzerland
3       Sunscreen                Aqualogica            259            400            India
-------------------------------------------------------------------------------------------------------------

Enter product ID to restock (0 to finish): -1
Invalid product ID! Please enter a valid ID from the list.

Enter product ID to restock (0 to finish): 1

Product: Vitamin C Serum
Brand:  Garnier
Current stock: 28
Cost price: Rs. 400
Enter quantity to add: -2
Quantity must be positive!

Enter product ID to restock (0 to finish):
```

Figure 38:NEGATIVE VALUE WHILE RESTOCKING

4.3. Test 3: File generation of purchase of product(s) (Purchasing multiple product(s))

Table 3:FILE GENERATION OF PURCHASE OF CUSTOMERS

Objective	To test the file generation of products while purchased by customers.
Action	• Run the program • Enter 2 to sell to customers • Select the valid product id • Select the valid quantity. • Enter the name and phone number
Expected Result	A invoice should be generated in shell and text file.
Actual Result	A invoice generated in shell and text file.
Conclusion	The test was successful.

```
                    WELCOME TO THE 'WE CARE' SKIN CARE COMPANY
_____________________________________________________________________________


                Insert one of the following command to connect with WE CARE


1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 2
-------------------------------------------------------------------------------------------------------
ID       Product Name           Brand          Available Quantity      Price          Country Of origin
-------------------------------------------------------------------------------------------------------
1       Vitamin C Serum         Garnier              28              800            France
2       Skin Cleanser           Cetaphil             50              1000           Switzerland
3       Sunscreen               Aqualogica           259             400            India
-------------------------------------------------------------------------------------------------------
Enter the product ID that you want to buy (0 to exit): 1
How much would you like to buy: 3
Do you want to buy more product? (yes/no): yes
Enter the product ID that you want to buy (0 to exit): 1
How much would you like to buy: 3
Do you want to buy more product? (yes/no): yes
Enter the product ID that you want to buy (0 to exit): 2
How much would you like to buy: 2
Do you want to buy more product? (yes/no): no
```

Figure 39:INPUT TO SELL TO CUSTOMERS

```
=== CUSTOMER DETAILS FOR INVOICE ===
Enter customer name: mahima
Enter phone number: 9876543212


                  === Invoice of purchase ===


Customer: mahima                          Date: 2025-05-14 02:05:12.764214
Phone: 9876543212

-------------------------------------------------------------------
        Product                                  Vitamin C Serum
        Quantity                                 6
        Brand                                     Garnier
        Free                                     2
        Unit Price                               Rs. 800
        Product                                  Skin Cleanser
        Quantity                                 2
        Brand                                     Cetaphil
        Free                                     0
        Unit Price                               Rs. 1000
-------------------------------------------------------------------
        Total                                    2000

-------------------------------------------------------------------

Invoice saved as: Bill_for C mahima_2025-5-14.txt
Total amount: Rs. 6800
Thank you for your purchase!
```

Figure 40:INVOICE IN SHELL

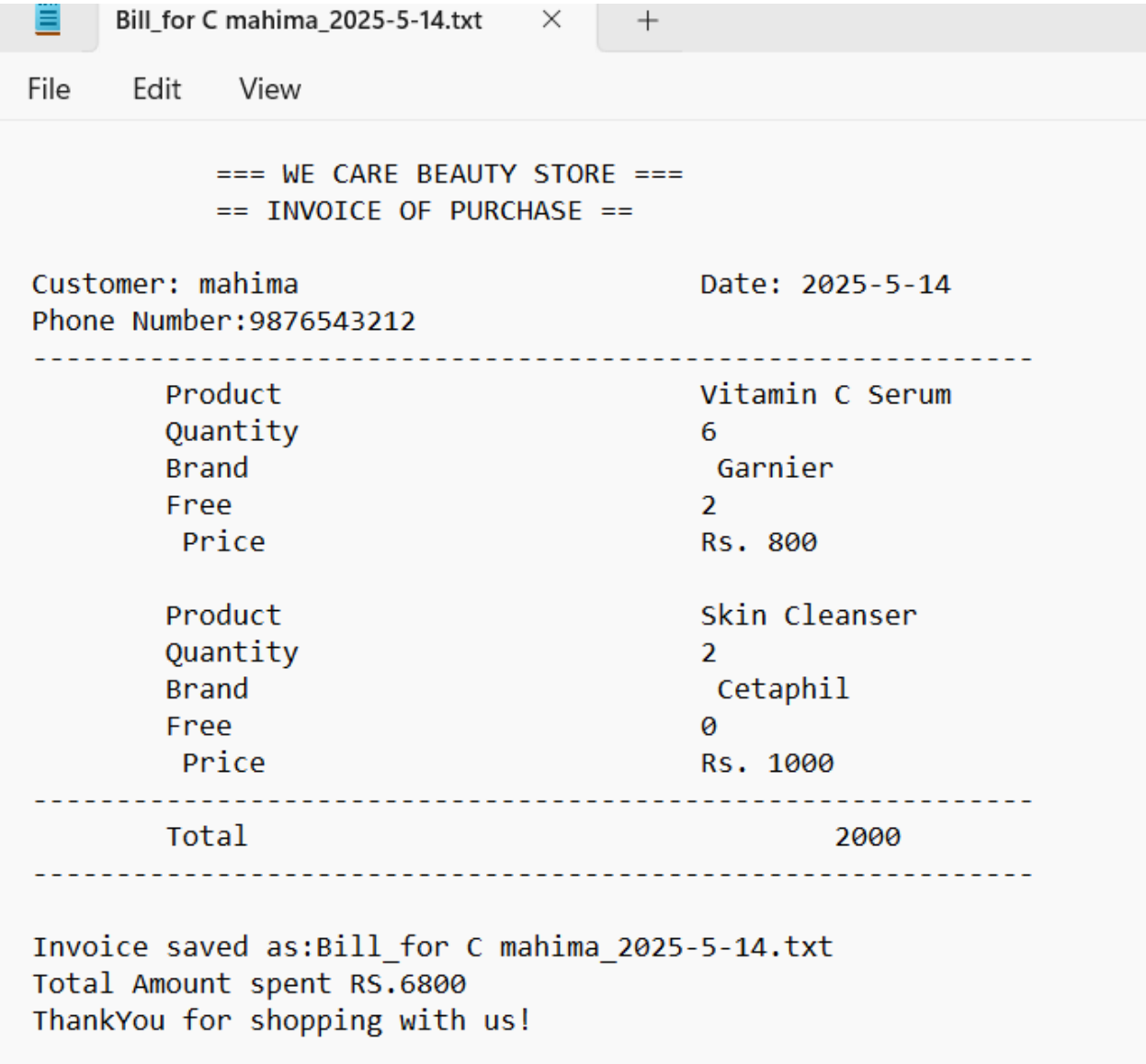


Figure 41:INVOICE IN .TXT

4.4. Test 4: File generation of sales process of product(s) (Selling multiple product(s))

table 4:RESTOCK PRODUCTS

Objective	To test the file generation of products while purchased by customers.
Action	• Run the program • Enter the product id • Enter the valid quantity • Enter the name and phone number
Expected Result	An invoice should be generated in the shell and the txt file.
Actual Result	An invoice generated in the shell and the txt file.
Conclusion	The test was successful.

```
1. Display the available products
2. To Sell from WE CARE to customer
3. To restock products from Supplier
4. To exit from the program

Please choose an option (1-4): 3
-------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand           Available Quantity      Price           Country Of origin
-------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum          Garnier                 50              800             France
2       Skin Cleanser            Cetaphil                70              1000            Switzerland
3       Sunscreen                Aqualogica              260             400             India
-------------------------------------------------------------------------------------------------------------

Enter product ID to restock (0 to finish): 3

Product: Sunscreen
Brand:  Aqualogica
Current stock: 260
Cost price: Rs. 200
Enter quantity to add: 5
Enter new cost price (current: Rs. 200), or press Enter to keep same:

Successfully added 5 units of Sunscreen
New stock level: 265
Do you want to RESTOCK more product? (yes/no): yes

Enter product ID to restock (0 to finish): 1

Product: Vitamin C Serum
Brand:  Garnier
```

Figure 42:INPUT FOR RESTOCK

```
Product: Vitamin C Serum
Brand:  Garnier
Current stock: 50
Cost price: Rs. 400
Enter quantity to add: 5
Enter new cost price (current: Rs. 400), or press Enter to keep same:

Successfully added 5 units of Vitamin C Serum
New stock level: 55
Do you want to RESTOCK more product? (yes/no): no

=== SUPLIER DETAILS FOR INVOICE ===
Enter Vendors name: ritu
Enter phone number: 9786765456
```

Figure 43:

```
                        === SUPPLIERS DETAILS ===


Supplier Name : ritu                    Date: 2025-05-14 02:42:29.704124
Phone: 9786765456

--------------------------------------------------------------------------
        Product                                 Sunscreen
        Brand                                    Aqualogica
        Qty                                     5
        Unit Price                              Rs. 200
        Product                                 Vitamin C Serum
        Brand                                    Garnier
        Qty                                     5
        Unit Price                              Rs. 400
--------------------------------------------------------------------------
        Total                           Rs. 3000
--------------------------------------------------------------------------


TOTAL COST: Rs. 3000
Total Cost After VAT 3390.0

Invoice saved as: Bill for Sritu_2025-5-14.txt

Restocking complete.
```

Figure 44:INVOICE IN SHELL

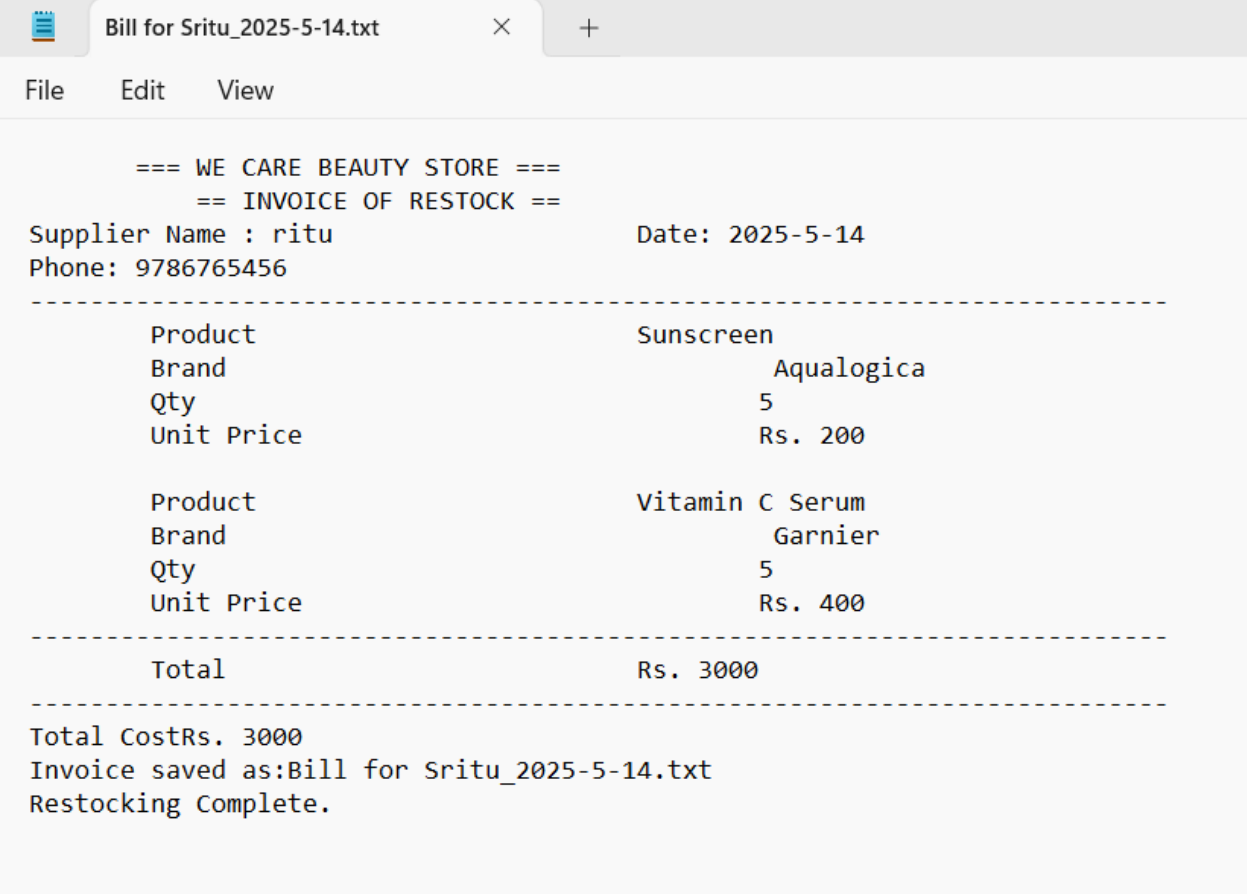


Figure 45:INVOICE IN.TXT

4.5. Test 5: Showing the update in the stock of product(s)

Table 5:UPDATE STOCK OF PRODUCTS

Objective	To show deduction in quantity while selling to the customers and added while restocking in .txt file
Action	For selling to customer • Run the code • Enter 2 to sell • Enter 1 for product id • Enter 5 for quantity For restocking • Run the code • Enter 3 to restock • Enter 1 for product id • Enter 1 for quantity
Expected Result	At the time of selling the quantity should deducted and while restocking should be quantity added.
Actual Result	At the time of selling the quantity deducted and while restocking quantity added.
Conclusion	The test was successful.

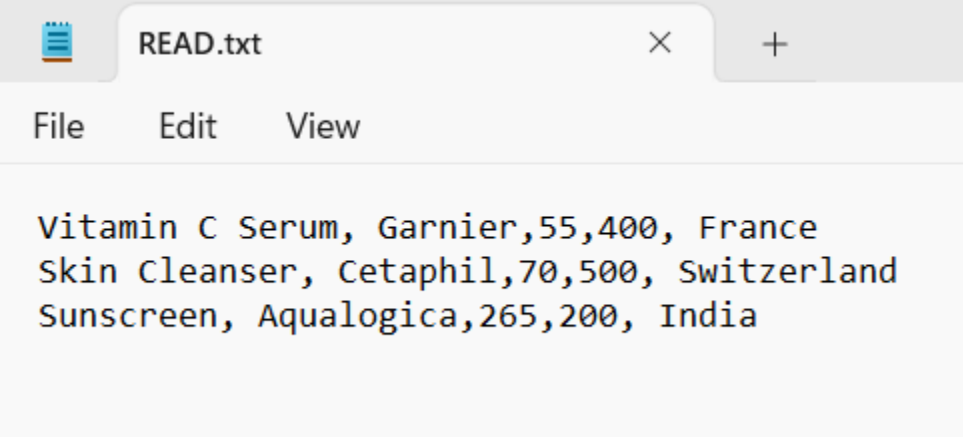


Figure 46:READ VALUE

```
-------------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand          Available Quantity      Price          Country Of origin
-------------------------------------------------------------------------------------------------------------------
1        Vitamin C Serum         Garnier               55              800            France
2        Skin Cleanser           Cetaphil              70              1000           Switzerland
3        Sunscreen               Aqualogica            265             400            India
-------------------------------------------------------------------------------------------------------------------
Enter the product ID that you want to buy (0 to exit): 1
How much would you like to buy: 5
Do you want to buy more product? (yes/no): no

=== CUSTOMER DETAILS FOR INVOICE ===
Enter customer name: gvbh
Enter phone number: 3456


                  === Invoice of purchase ===


Customer: gvbh                  Date: 2025-05-14 02:56:39.634403
Phone: 3456

-----------------------------------------------------------------
        Product                                 Vitamin C Serum
        Quantity                                5
        Brand                                    Garnier
        Free                                    1
        Unit Price                              Rs. 800
-----------------------------------------------------------------
        Total                                   4000
```

Figure 47:SELLING TO CUSTOMER

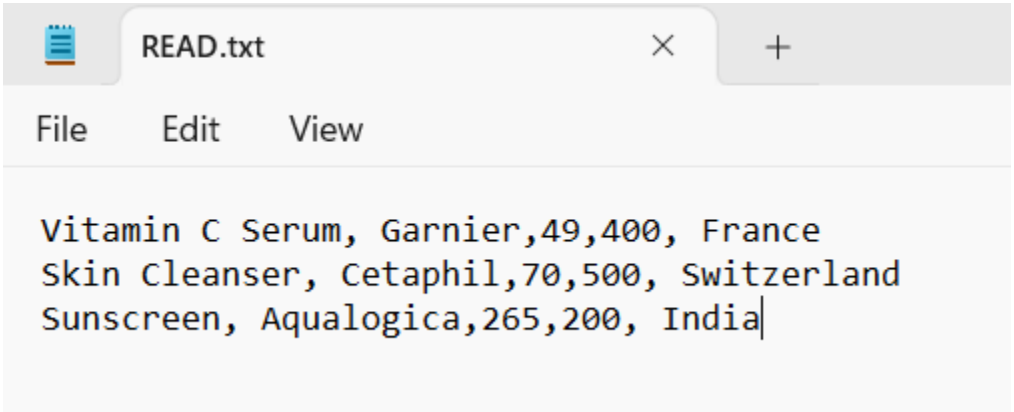


Figure 48:UPDATED DEDUCTED VALUE

```
Please choose an option (1-4): 3
-------------------------------------------------------------------------------------------------------------------
ID       Product Name            Brand           Available Quantity      Price           Country Of origin
-------------------------------------------------------------------------------------------------------------------
1       Vitamin C Serum          Garnier                49              800             France
2       Skin Cleanser            Cetaphil               70              1000            Switzerland
3       Sunscreen                Aqualogica             265             400             India
4
-------------------------------------------------------------------------------------------------------------------

Enter product ID to restock (0 to finish): 1

Product: Vitamin C Serum
Brand:  Garnier
Current stock: 49
Cost price: Rs. 400
Enter quantity to add: 1
Enter new cost price (current: Rs. 400), or press Enter to keep same:

Successfully added 1 units of Vitamin C Serum
New stock level: 50
Do you want to RESTOCK more product? (yes/no): no

=== SUPLIER DETAILS FOR INVOICE ===
Enter Vendors name: fgh
Enter phone number: 45678
```

Figure 49:RESTOCKING PRODUCTS

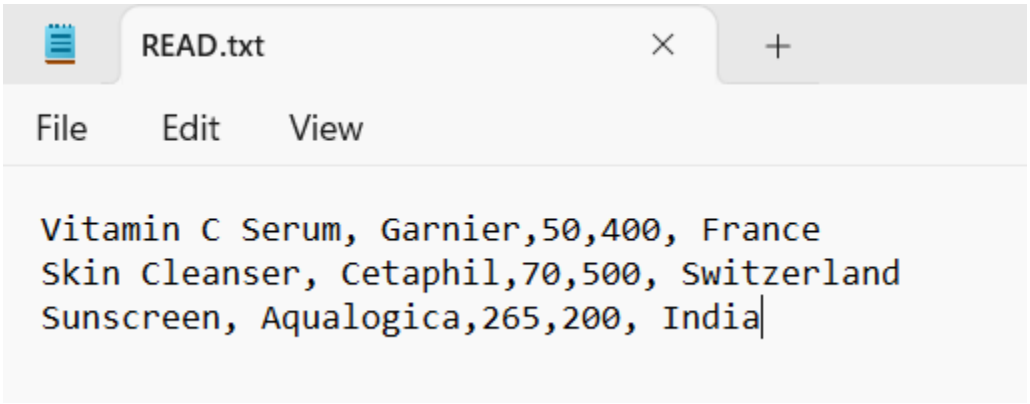


Figure 50:UPDATED ADDED VALUE

5. Conclusion

The development of the system of the ‘WE CARE’ has been successfully complete.this project’s major objective was to develop a user frieldly system for the store we care so the buying and selling would be a lot more easier. In this project a helpful technology was implemented by the WE CARE store by which they can benefit the customes as well as them by saving a lot of time and effort and also providing great deals to the customers.

To summon up the experience of the development of this project was very knowledgeable, and I get to learn a lot of about the python programming.in this project we learne to make a functulable program through python which can take input display available products,sell to the customers and restock from suppliers very efficiently and accurately.

Through this course the knowledge of the python programming has been a lot better .i got to explore on the new topics,learnes exceptional handlings and got a bit of exposure of how the logic works inoder to make a program.The creation of the program was itself very scary yet fun experience from not knowing anything to have some of the idea of the algorithm of the program has enhanced the knowledgeof python programming.

In conclusion,this project of creating a program has helped to have strong foundation for digital store management skills and getting to use our studies in reall ife program helps to make us understand better and improve the current potential of creativeness in us.with the help of the program it benefits the store and the and with further development plan the system can be expanded to be more better and we can learn a lot more from the system to make it much better.

6.Bibliography

GeeksforGeeks, 2023. *What is Python?* [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/what-is-python/ [Accessed 19 Apr. 2025].

GeeksforGeeks, 2021. *Introduction to Microsoft Word*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/introduction-to-microsoft-word/ [Accessed 19 Apr. 2025].

draw.io. (n.d.). *About*. Retrieved May 4, 2025, from https://www.drawio.com/about

7. Appendix

7.1. Code of READ.py

```
import datetime
'''this READ.py file allows all the file in this program to display the
available stock of the shop and displays the imformation to the user
for buying and restocking'''
def reading():
    '''Summary:
        this function reads the data from'READ.txt', and displays
        it in a tabular format with calculated seling price
        and returns the data as a dictionary

    Parameters:None

    Returns:
        it returns disc_ where the key is product ID(int),
        and list containing:
        [Product Name(str),Brand(str),Available Quantity(str),Price(str),Country Of origin(str)].
    Raises:
        FileNotFoundError:if 'READ.txt'file doesnot exist.

        '''
#read the existing document
    print("-" * 110)
    print("ID","\t","Product Name","\t\t","Brand","\t\t","Available Quantity","\t","Price","\t\t","Country Of origin")
    print("-" * 110)

#To read from the txt file
    with open('READ.txt', 'r') as file:
```

```
        store = file.read()

    listen = store.split('\n')#splits the column
    file.close()
    disc_ = {}

    for i in range(len(listen)):
        disc_[i + 1] = listen[i].split(',')#splits according to ,.

    for key, value in disc_.items():
        print(key, end='\t')

        for i in range(len(value)):
            if i == 3:#the selling price calculation
                cost_price = int(value[i])
                selling_price = cost_price * 2
                print(selling_price, end='\t\t')
            elif i == 0:#to make the indentation correct
                print(value[i], end='\t\t')
            else:#to make the indentation correct
                print(value[i], end='\t\t')
        print()
    print('-' * 110)

    return disc_
```

7.2. Code of WRITE.py

```
import datetime
def bill_for_customers(items_bought,customer_name,phone_number,amt_spent):
    '''
    Summary:This file generate and saves a text invoice of a customer
    if they purchase anything.
    the invoice is saved in txt file where customer details and all the
    necessary details are stored.

    Parameters:
    items_bought(list of dictionary):In this list following information is included:
        'name'(Str):name of the product
        'quantity(int)':quantity purchased
        'brand'(str):brand of the product bought
        'free'(int):number of free item if got any
        'price'(int):price per item
        'total'(int):total price after free gifts
    customer_name(str):Customer's name
    phone_number(str):Cuctomer's phone number
    amt_apent(int):Total amount customer spent on total

    Returns:
    str:fileName (str) of the invoice which is stored on disk.

    Raises:
    TypeError:If incorrect data types are passes for any parameter.
'''
    now = datetime.datetime.now()
    date = str(now.year) + "-" + str(now.month) + "-" + str(now.day)
    fileName="Bill_for C "+customer_name+"_"+date+".txt"
```

```
    with open (fileName,'w')as file:
        #to print the invoice of customers
        file.write("          === WE CARE BEAUTY STORE ===\n")
        file.write("\t   == INVOICE OF PURCHASE ==\n")
        file.write("\n")
        file.write("Customer: " + customer_name + "\t\t\tDate: "+date+"\n" )
        file.write("Phone Number:"+phone_number+"\n")

        file.write("-"*60)
        #writing in the txt file
        for item in items_bought:
            #customers purchase
            file.write("\n")
            file.write("\tProduct"+"\t"*4+item['name'] + "\n")
            file.write( "\tQuantity"+"\t"*3+str(item['quantity']) + "\n" )
            file.write( "\tBrand"+"\t"*4+item['brand']+ "\n")
            file.write( "\tFree"+"\t"*4+ str(item['free']) + "\n" )
            file.write( "\t Price"+"\t"*4+"Rs. "+str(item['price']))
            file.write("\n")
        file.write("-"*60)
        file.write("\n")
        file.write("\tTotal"+"\t"*5 +str(item['total'])+"\n")
        file.write("-"*60)
        file.write("\n")
        #saving the invoice with the user's name and date
        file.write("\nInvoice saved as:"+fileName)
        file.write("\n")
        #displays how much amount is spent
        file.write("Total Amount spent RS."+ str(amt_spent) +"\n")
        file.write("ThankYou for shopping with us!\n")
```

```
    return fileName

def bill_for_vendors(restocked_items,total_cost,supplier_name,phone_number):
    ''' Summary:
            This function generates and saves a text invoice
            to restock items in the store.
            this invoice includes supplier details,list of items restocked
            with its price and quantity.
        Parameters:
            restocked_items(lists of dict): it is list of dictionary
            which contains following information:
                'name'(str):name of the product
                'brand'(str):brand of the product
                'quantity'(str):quantity bought
                'cost_price'(int):original price
                'total'(int):total anount together
            total_cost(int):total cost of all the items
            supplier_name(str):name of the supplier]
            phone_number(str):phone number of vendor

        Returns:
            str:filename(str) of the invoice file is create and is saved in disks.
        Raises:
            TypeError:if any of the parameter is passes with incorrect type.

)

'''
    now = datetime.datetime.now()
    date = str(now.year) + "-" + str(now.month) + "-" + str(now.day)
```

```
    filename="Bill for S"+ supplier_name+"_"+date+".txt"
#opens a new file in vendor's name and date
    with open(filename,'w')as file:
        #format for the display
        file.write("       === WE CARE BEAUTY STORE ===\n")
        file.write("\t    == INVOICE OF RESTOCK ==\n")
        file.write("Supplier Name : " + supplier_name + "\t\t\tDate: " +date+"\n")
        file.write("Phone: " + phone_number + "\n")
        file.write("-"*75)
        for item in restocked_items:
            #necessary information to display  restock
            file.write("\tProduct"+"\t"*4+item['name'] + "\n")
            file.write("\tBrand"+"\t"*5  +item['brand'] + "\n")
            file.write("\tQty"+"\t"*5+ str(item['quantity']) + "\n" )
            file.write("\tUnit Price" +"\t"*4+"Rs. " + str(item['cost_price']))
            file.write("\n" )
        file.write("-"*75)
        file.write("\n" )
        file.write("\tTotal"+"\t"*4+"Rs. " + str(item['total']) +"\n")
        file.write("-"*75)
        file.write("\n")
        #total cost spent
        file.write("Total Cost"+"Rs. " + str(item['total']))
        # writes saves the invoice as
        file.write("\nInvoice saved as:"+filename)
        file.write("Restocking Complete.\n")

    return filename
```

7.3. Code of OPERATION.py

```
from READ import reading
import datetime
from WRITE import bill_for_customers, bill_for_vendors


def selling():
    '''
    Summary:
        This function handles the selling to the customers.It allows the user
        to purchase product and they can also get "buy 3 get 1 free"offer,
        updates the product,takes the necessary information fron the user and also
        displays the invoice using 'biil_for_customers' function.

    Parameters:
        None

    Returns:
        None

    Raises:
        ValueError:If non integer value is entered in product id or quantity.
        FileNotFoundError:If 'READ.txt' does not exist os is not accessible.
'''
    all_the_products = reading()
    items_bought = []  # This will store bought items
    amt_spent = 0

    while True:
        try:
            #take input of the product id
            product_id = int(input("Enter the product ID that you want to buy (0 to exit): "))
```

```
if product_id == 0:
    break

if product_id not in all_the_products:
    print("Enter a valid product id available!")
    continue

product = all_the_products[product_id]
product_name = product[0]
brand = product[1]
available_qty = int(product[2])
price = int(product[3]) * 2  # Adjusted for price logic

# Check if product is out of stock
if available_qty <= 0:
    print("This product is out of stock. Please choose another product.")
    continue

while True:
    try:
        qty_to_buy = int(input("How much would you like to buy: "))
        #checks the quantity is vaild or available or not.
        if qty_to_buy <= 0:
            print("Invalid quantity.")
            continue
        if qty_to_buy > available_qty:
            print("The available quantity is:", available_qty)
            continue
        #for the item alredy inserted
        found = False
```

```
for item in items_bought:
    if item['id'] == product_id:
        item['quantity'] += qty_to_buy
        total_qty = item['quantity']
        item['free'] = total_qty // 3
        paid_items = total_qty + item['free']
        item['total'] = qty_to_buy * price
        found = True
        break
#if it is not found update here
if not found:
    total_qty = qty_to_buy
    free_items = total_qty // 3
    paid_items = total_qty + free_items
    item_total = qty_to_buy * price

    # Update the product info in the list
    items_bought.append({
        'id': product_id,
        'name': product_name,
        'brand': brand,
        'quantity': qty_to_buy,
        'free': free_items,
        'price': price,
        'total': item_total
    })

# Updating the quantity in the inventory
new_qty = available_qty - paid_items
if new_qty < 0:
    new_qty = 0
```

```
                product[2] = str(new_qty)

                amt_spent += item_total
                break

            except ValueError:
                print("Please enter a valid number!")

        with open('READ.txt', 'w') as file:
            for pid in sorted(all_the_products.keys()):
                product = all_the_products[pid]
                # Write back to file in the original format
                file.write(','.join(product) + '\n')

        while True:
            #if the user want to continue buying
            again = input("Do you want to buy more product? (yes/no): ").lower()
            if again in ('yes', 'no'):
                break
            print("Please enter 'yes' or 'no'")

        if again == 'no':
            break

    except ValueError:
        print("Error: Please enter a valid number!")
        continue

if not items_bought:
    print("\n No items were purchased.")
    return
```

```
# Get customer details for invoice
    print("\n=== CUSTOMER DETAILS FOR INVOICE ===")
    while True:
        customer_name = input("Enter customer name: ")
        if(customer_name.isalpha()):
            break
        else:
            print("Name not valid.")
    while True:
        phone_number = input("Enter phone number: ")
        if(phone_number.isdigit()):
            break
        else:
            print("Enter a vaild phone number")


filename=bill_for_customers(items_bought,customer_name,phone_number,amt_spent)
    print("\n")
    print("\t\t  === Invoice of purchase ===")
    print("\n")
    print("Customer: " + customer_name + "\t\t\t","Date: " + str(datetime.datetime.now()) )

    print("Phone: " + phone_number + "\n")
    print("-" * 65)
    for item in items_bought:
        print("\tProduct","\t"*4+item['name'] + "\n" + "\tQuantity","\t"*4+str(item['quantity']) +
"\n" + "\tBrand","\t"*5+str(item['brand']) + "\n" +"\tFree","\t"*5+ str(item['free']) + "\n" +
                "\tUnit Price","\t"*4+"Rs. "+str(item['price']))
    print("-" * 65)
    print("\tTotal","\t"*5 +str(item['total'])+"\n")
```

```
    print("-" * 65)

    print("\nInvoice saved as:", filename)
    print("Total amount: Rs.", amt_spent)
    print("Thank you for your purchase!")
    print("\n"*5)

def restocking():
    '''
    Summary:
        This function handles the restocking preocess of the WE CARE.
        It allows the user to update quantity ,cost price and calculates vat
        and generates invoice using 'bill_for_vendors()' function.
    Parameters:
        None
    Returns:
        None
    Raises:
        ValueError:It is raised if non numeric input is provided in product id,quantity or price.
        FileNotFound:If 'READ.txt' does not exist or is inaccessible.
'''
    products = reading()
    restocked_items = []
    total_cost = 0

    while True:
        try:
            product_id = int(input("\nEnter product ID to restock (0 to finish): "))

            if product_id == 0:
                break
```

```
        if product_id not in products:
            print("Invalid product ID! Please enter a valid ID from the list.")
            continue

        product = products[product_id]
        current_qty = int(product[2])
        cost_price = int(product[3])   # Gets original cost price

        print("\nProduct: " + product[0])
        print("Brand: " + product[1])
        print("Current stock: " + str(current_qty))
        print("Cost price: Rs. " + str(cost_price))

        try:
            #Validation of the quantity is checked
            restock_qty = int(input("Enter quantity to add: "))
            if restock_qty==0:
                break
            if restock_qty <= 0:
                print("Quantity must be positive!")
                continue
            #to calculate new price if the price has changed
            new_cost_price = input("Enter new cost price (current: Rs. " + str(cost_price) +
"), or press Enter to keep same: ")

            if new_cost_price:
                try:
                    new_cost_price = int(new_cost_price)
                    if new_cost_price <= 0:
                        print("Price must be positive! Keeping original price.")
                    else:
```

```
                cost_price = new_cost_price
                product[3] = str(cost_price)
            except ValueError:
                print("Invalid price! Keeping original price.")

        # Calculate item total
        item_total = restock_qty * cost_price
        total_cost += item_total
        amt_after_vat=(total_cost*0.13)+total_cost

        # Add to restocked items list
        restocked_items.append({
            'name': product[0],
            'brand': product[1],
            'quantity': restock_qty,
            'cost_price': cost_price,
            'total': total_cost
        })

        # Update quantity
        product[2] = str(current_qty + restock_qty)

        # Write changes back to file
        with open('READ.txt', 'w') as file:
            for pid in sorted(products.keys()):
                p = products[pid]
                file.write(','.join(p) + '\n')

        print("\nSuccessfully added " + str(restock_qty) + " units of " + product[0])
        print("New stock level: " + product[2])
```

```
            while True:
                #if they want to restock more
                again = input("Do you want to RESTOCK more product? (yes/no): ").lower()
                if again in ('yes', 'no'):
                    break
                print("Please enter 'yes' or 'no'")

            if again == 'no':
                break
        except ValueError:
            print("Invalid quantity! Please enter a number.")

    except ValueError:
        print("Invalid input! Please enter a number.")
if not restocked_items:
    print("\n No items were RESTOCKED.")
    return
#display format for invoice
print("\n=== SUPLIER DETAILS FOR INVOICE ===")
supplier_name = input("Enter Vendors name: ")
while True:
    phone_number = input("Enter phone number: ")
    if(phone_number.isdigit()):
        break
    else:
        print("Enter a valid number.")
#prints the invoice for supplier
if restocked_items:

    filename                                                    =
bill_for_vendors(restocked_items,total_cost,supplier_name,phone_number)
```

```
        print("\n")
        print("\t\t\t=== SUPPLIERS DETAILS ===")
        print("\n")
        print("Supplier Name : " + supplier_name + "\t\t\t","Date: " +
str(datetime.datetime.now()) )
        print("Phone: " + phone_number + "\n")

        print("-" * 75)
        for item in restocked_items:
            print("\tProduct","\t"*4+item['name'] + "\n" +"\tBrand","\t"*5  +item['brand'] + "\n"
+"\tQty","\t"*5+ str(item['quantity']) + "\n" +
             "\tUnit Price" ,"\t"*4+"Rs. " + str(item['cost_price']))
        print("-" * 75)
        print("\tTotal","\t"*4+"Rs. " + str(item['total']))
        print("-" * 75)
        print("\n")
        print("TOTAL COST: Rs.", total_cost)
        #vat calculation
        print("Total Cost After VAT",(str(amt_after_vat)))
        print("\nInvoice saved as:", filename)
        print("\nRestocking complete.")
        print("\n" *15)
```

7.4. Code of MAIN.py

```
from OPERATION import selling,restocking
from READ import reading

def main_class():
    """
    Summary:
        this function display the main menu of the WE CARE system and also
        handels user interaction.
        there are option to display,sell to customers,restock or exit the
        program.

    Parameters:
        None

    Returns:
        None

    Raises:
        ValueError:If any non numeric values are given when number
        should be (1-4).
"""
    while True:
        print("*-*"*35)
        print("\n\t\t\t*=== ---------- WE CARE ---------- ===*")
        print("*-*"*35)
        print("\n")
        print("\t\t\t   WELCOME TO THE 'WE CARE' SKIN CARE COMPANY")
        print("_"*105)
        print("\n")
```

```
    print("\t\t  Insert one of the following command to connect with WE CARE")
    print("\n")
    print("1. Display the available products")
    print("2. To Sell from WE CARE to customer")
    print("3. To restock products from Supplier")
    print("4. To exit from the program")

    user_choice = input("\nPlease choose an option (1-4): ")

    if user_choice == '1':#to display the file
        reading()
    elif user_choice == '2':#selling to the customers
        selling()
    elif user_choice == '3':#restoking from the vendor
        restocking()
    elif user_choice == '4':
        #user input to make sure
        decision = input("Are you sure you want to exit? (yes/no): ").lower()
        if decision == 'yes':
            print("\n\n\nThank you for choosing WE CARE. Hope you had a great
experience here!!")
            break
        elif decision == 'no':
            continue
        else:
            print("Invalid input! Please enter 'yes' or 'no'")
    else:
        print("\nInvalid choice! Please choose options 1-4")

main_class()
```